参赛作品 · 技术材料

智能日程助手

源 码 解 析

一份关于本地优先日程应用的实现说明：从数据模型、仓储与接口，到有界 AI Agent、 自研富文本排版，再到三平台安装包的构建链路。全文以当前仓库代码为准，逐条标注文件与行号。

54	9,454	21	9
源文件	行 TypeScript	HTTP 接口	Agent 工具

本地优先 | 有界 Agent | 自研排版 | 三平台交付

Next.js 14.2.35 · React 18 · TypeScript 5.5

版本 v1.2.0 | 2026-10-26

目 录

页码对应正文实际页序；章节以“第 N 章”标识，附录以 A/B/C 标识。

第 1 章 **文档说明与代码基线** 3

读者与范围、代码基线、统计口径与引用约定、术语表

第 2 章 **技术栈与目录结构** 6

选型与理由、目录分层、接口总览、前后端共用边界

第 3 章 **类型与数据模型** 9

Todo 字段、优先级与权重、历史数据收敛、状态机、云库映射

第 4 章 **仓储层：本地文件与云适配** 12

一行分发、JSON 仓储、三处真实局限、自动维护、云库代价

第 5 章 **接口层设计** 14

路由组织、五分支批量端点、错误码约定、运行配置

第 6 章 **AI 能力层** 16

统一调用、抽取提示词、错误中文化、四道校验、无 Key 兜底

第 7 章 **Agent 设计** 19

有界循环、九个工具、停止条件、JSON 降级、人在环删除与撤销、只读巡检

第 8 章 **前端架构与富文本排版** 26

状态管理、五视图、交互细节、RichText 排版引擎

第 9 章 **导入与导出** 29

双模式判定、表格解析、文档与图片、三种导出、分享长图

第 10 章 **三平台打包与本地化** 32

standalone 构建、SEA 注入、自解压安装、数据外置、端口探活、平台差异

第 11 章 **工程决策与踩坑记录** 36

八个决策的代价、易错点表、可复现验收清单

第 12 章 **二次开发指南** 38

新增 Agent 工具、替换存储、新增导入格式

第 13 章 **增补：后台提醒与覆盖安装** 41

提醒时间线、送达链路、清理五步、两轮自检结果

第 14 章 **多主题系统：一份组件，四套皮肤** 44

四套主题的取舍、变量与重映射、星光实现、无障碍与自检

附 录 **接口全表 / 环境变量 / 文件清单** 47

A 接口一览、B 环境变量、C 文件与行数

01 文档说明与代码基线

SCOPE · BASELINE · CONVENTIONS

> 这一章说明三件事：这份文档写给谁看、解析的是哪一份代码、文中出现的引用与统计是怎么算出来的。先把边界讲清楚，后面的结论才有意义。

1.1 读者与范围

本文面向两类读者。一类是评委与评审，希望在不运行程序的前提下判断这个作品的技术含量； 另一类是准备接手或二次开发的人，需要知道某个行为到底写在哪一行。 因此全文采用同一种写法：先给结论，再给实现位置，最后说清代价与取舍。

解析范围是应用本身的源代码，包括领域模型、仓储、接口、AI 能力、Agent 运行时、前端组件与富文本渲染。 打包脚本（Node SEA 启动器、自解压安装器、macOS 与 Linux 产物生成）虽然不在 `src` 目录内， 但决定了「用户拿到的到底是什么」，因此单列一章。第三方依赖、Next.js 框架内部实现、 以及操作系统层面的行为不在解析范围内，只在必要处作为背景出现。

1.2 代码基线

本文以仓库当前代码为准，统计口径为 `src/` 下 54 个文件、9,454 行 TypeScript（含 tsx）， 不含 `node_modules`、构建产物与运行期数据。这一版对应 v1.2.0（在 v1.0 基础上增补后台提醒、覆盖安装与多主题）， 与随作品提交的源码包、三个平台的安装包同源。

需要说明一个过程事实：最早的源码包是在 10 月 23 日 11:31 打的快照，此后又迭代了四轮， 主要补齐了富文本排版引擎、健康探针、导入双模式与表格解析。 两份代码逐文件比对后，差异集中在 11 个文件、约 28 KB：

文件	旧快照	当前	变化
`src/lib/server/agent/tools.ts`	16.3 KB	25.3 KB	新增三个工具与批量执行
`src/components/ImportModal.tsx`	11.9 KB	15.6 KB	导入双模式与手动输入
`src/components/RichText.tsx`	—	7.0 KB	新增，全文排版引擎
`src/app/api/import/parse/route.ts`	15.0 KB	17.2 KB	mode 分支与 Excel 转文本
`src/lib/server/agent/runtime.ts`	18.4 KB	19.4 KB	结构化输出约束、只读模式
`src/app/api/health/route.ts`	—	0.4 KB	新增，启动器指纹探活

表 1-1 旧快照与当前代码的主要差异（其余 6 个文件为小幅增量）

提交的源码包已按当前代码重新打包，并补入了 `build-desktop/` 与 `installer/` 两个目录， 使第 10 章的每一条结论都能在包内找到对应文件。阅读时若发现文档与代码不一致，以代码为准。

1.3 统计口径与引用约定

文中引用统一写作 `路径:起始行-结束行`，例如 `src/lib/server/repository.ts:40-45`， 表示结论来自该文件的这一段。行号以当前仓库为准，若后续改动导致偏移，以函数名或代码片段为准检索。

代码片段均为原文摘录，只做两件事：省略与结论无关的分支、把过长的注释折叠为一行。 涉及敏感值的地方（API Key、用户目录）一律以占位符出现。数字类结论（行数、体积、超时阈值） 都标注了来源，没有来源的数字不写。

1.4 术语表

术语	在本文中的含义
Agent	由模型选择工具、由本地控制器执行的有界循环。模型不直接写数据库，只产出工具调用意图。
HITL	Human-in-the-loop。删除类操作只生成待确认清单，必须由用户在界面上二次确认才落库。
有界循环	步数、墙钟、单次调用超时、Token 预算四重上限，任一触顶即停止并给出可读原因。
JSON 降级	模型或网关不支持 function calling 时，改用「一步一个 JSON 对象」的文本协议继续规划。
standalone	Next.js 的独立部署产物，把运行期依赖收进 `.next/standalone`，只需一个 Node 进程即可启动。
SEA	Node.js Single Executable Application，把 JS 与资源注入 node 可执行文件，得到单文件程序。
junction	Windows 目录联接。用来让 `app/data` 指向安装目录外层的 `data`，升级时不丢数据。

1.5 全文结构

十二章加一个附录，可以按「是什么、怎么做、为什么」三段读。下表是每章要回答的问题与核心结论， 赶时间的话看这一张表即可。

章	回答的问题	核心结论
1	解析的是哪份代码	与提交的源码包、三个安装包同源；旧快照差 11 个文件
2	分几层，边界在哪	五层；前端到接口只有 `api-client` 一个出口
3	一条日程怎么表达	21 字段 + 三态 + 软删除；`isDeadline` 区分长时段
4	数据存在哪	JSON 全量读写，三处已知局限；自动维护顺序不可换
5	前后端怎么说话	19 个 handler；错误码本身就是引导
6	模型怎么被调用	请求级 Key 优先；四道清洗；无 Key 时明确降级
7	模型被允许做什么	6 步 / 90 秒 / 30k；删除只提议；写操作可撤销
8	界面怎么组织	单 store 29 个动作；自研 RichText 统一层级
9	数据怎么进出	导入双模式；导出三种格式；长图借系统浏览器
10	怎么装到别人电脑	standalone + SEA + 自解压 + 数据外置 + 端口指纹
11	为什么这样选	八个交换、十个坑、一份可复现验收清单
12	接下来怎么改	工具、存储、格式三处固定挂载点
13	提醒与覆盖安装怎么实现	60/180 分钟两条时间线；清理五步与三道护栏
14	多主题怎么做到一份组件	CSS 变量 + 作用域重映射；四套皮肤共用交互

表 1-2 章节索引

三条阅读路线：想评估技术含量，读第 7、10 章，那里是自研成分最重的部分； 想接手开发，读第 4、5、12 章，改动点最集中； 只想知道这个作品解决什么问题，读第 8 章与第 9 章的开头两段即可。

02 技术栈与目录结构

STACK · LAYOUT · BOUNDARIES

> 这个应用要同时满足三个约束：单机可运行、界面要有质感、AI 能力要能落地。 技术选型基本是被这三条推着走的结果，而不是先挑框架再找场景。

2.1 选型与理由

运行形态是「一个本地进程 + 浏览器界面」。没有选择 Electron，原因在 2.5 节单独说明。 服务端用 Next.js 的 App Router，接口与页面在同一工程内，构建产物是 standalone 目录， 可以直接被一个 Node 进程拉起。前端状态用 Zustand，原因是它没有 Provider 嵌套， 在抽屉、弹窗、视图之间共享筛选条件时心智负担最小。

依赖	版本	承担的角色
`next`	14.2.35	页面与接口同构、standalone 构建、路由约定
`react` / `react-dom`	18.3.1	视图层
`zustand`	4.5.5	全局状态：待办列表、筛选、弹层、批量选择
`framer-motion`	11.5.4	抽屉与卡片的进出场动画
`lucide-react`	0.447.0	图标，统一线宽
`xlsx`	0.18.5	Excel/CSV 读取与工作表转 CSV
`mammoth`	1.13.0	docx 转纯文本
`qrcode`	1.5.4	局域网访问二维码，本地生成 SVG

表 2-1 运行时依赖（`package.json:11-22`）。打包工具链另计，不进安装包。

一个刻意的取舍是：**不引入 UI 组件库**。日历网格、看板列、统计条形图都是手写的， 换来的是暗色主题下每个像素都可控——组件库的默认浅色底在深色背景上会露出白块， 第 8 章提到的下拉框问题就是这类冲突的典型。

2.2 目录结构

目录按「离用户多远」分层，越靠上越接近界面，越靠下越接近磁盘：

```
intelligent-todo/
├─ src/app/                  # 路由：页面 + 接口
│  ├─ page.tsx               # 首页只渲染 AppShell
│  ├─ layout.tsx             # 元信息、PWA manifest、图标
│  ├─ globals.css            # 暗色主题变量与原子样式
│  └─ api/                   # 14 个 route.ts，19 个 handler
├─ src/components/           # 界面组件（外壳、视图、抽屉、弹窗）
├─ src/lib/                  # 前后端共用：类型、课程规则、API 客户端
│  └─ server/                # 仅服务端：仓储、AI 公共层、Agent
│     └─ agent/              # runtime / tools / log
├─ db/schema.sql             # 云库建表脚本（可选）
├─ scripts/                  # 评测、图标生成、假模型服务
├─ public/                   # PWA 图标、manifest、Service Worker
├─ build-desktop/            # 启动器、SEA 配置、三平台打包脚本
└─ installer/                # 自解压安装器
```

图 2-1　目录结构（略去配置文件与依赖目录）

2.3 一次请求走过的四个环节

以「新建一条待办」为例，请求只经过四个环节：组件调用 store 的 action， store 调用 `api-client`，客户端打到 `/api/todos`，路由转给仓储落盘。 这条链路上没有第二套数据源，也没有本地缓存副本——所有视图读的都是同一个 store 数组。

关键约束写在 `src/lib/api-client.ts`：它是前端唯一的 HTTP 出口。 组件里不允许出现裸 `fetch`，所有错误在这一层被转换成 `Error` 并附上可读文案， 界面只负责显示。`api-client` 还顺带做了一件事——把用户填在设置面板里的大模型配置 自动挂到请求体上（`src/lib/api-client.ts:101-110`），这样服务端不需要读浏览器存储。

2.4 分层与边界

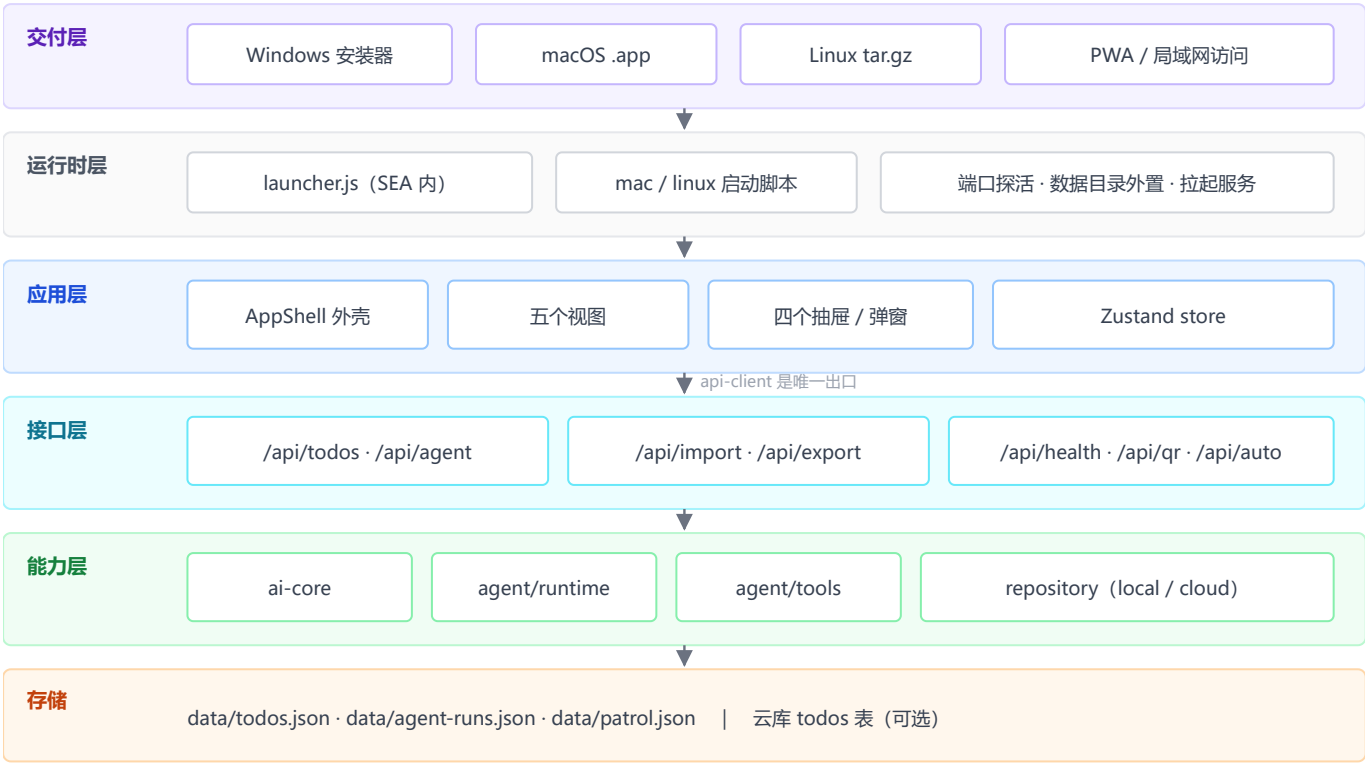


图 2-2 五层结构。应用层到接口层只有 `api-client.ts` 一条边；能力层到存储层的分叉由 `server/index.ts` 决定。

图中两条边界值得强调。第一条是应用层与接口层之间：所有请求经 `api-client` 收口， 带来的是错误处理与鉴权参数只写一次。第二条是 `src/lib` 与 `src/lib/server` 之间： 前者可以被客户端组件导入，后者一旦被客户端导入就会把 `fs` 打进浏览器包， 构建会直接报错。

唯一跨端复用的是 `types.ts` 与 `course.ts`。前后端都要判断「这是不是一门课」， 如果各写一份，规则迟早会漂移。把 `isCourseLike`、 `courseWindow`、 `courseStatusAt` 放在共用层，前端用来显示状态徽章，后端用来做自动流转，判断依据完全一致 （`src/lib/course.ts:9-58`）。

2.5 为什么不用 Electron

最直接的原因是体积与启动速度。Electron 方案即使什么都不做，安装包也在 80 MB 以上， 冷启动需要拉起一个完整 Chromium。这里选择的是「Node 进程 + 系统浏览器」： 界面由用户已有的 Edge/Chrome 渲染，安装包体积花在真正需要的地方（内置的无头浏览器只用于导出长图）。

代价也很明确：无法做系统级集成（托盘、开机自启、原生菜单），窗口就是普通标签页。 对这个作品来说这些能力不是刚需，而「装完就能用、不额外占内存」是刚需，所以这个交换是划算的。 第 10 章会看到，为了让这个形态在用户机器上稳定运行，反而要额外解决端口占用、数据目录、 升级覆盖三个 Electron 免费提供的问题。

03 类型与数据模型

MODEL · PRIORITY · STATE

> 整个应用只有一个实体：一条日程。课程、作业、会议、报名，都装进同一个结构里。 模型设计的难点不在于字段多少，而在于「哪些字段必须联动、哪些字段必须容错」。

3.1 一个模型承载三种形态

`Todo` 共 21 个字段（`src/lib/types.ts:9-42`）。按用途分成六组看，会清楚很多：

分组	字段	说明
身份	`id` `createdAt` `updatedAt`	时间戳用 ISO 字符串，前端直接切片显示
内容	`title` `notes` `tags[]`	标题 200 字上限，备注 500 字，标签最多 6 个
时间	`dueDate` `startDate` `dueTime` `endTime`	日期 `yyyy-mm-dd`，时间 `HH:mm`，均可为 null
形态	`isDeadline`	true 表示跨多日的长时段事项，此时 `startDate` 才有意义
状态	`done` `status` `completedAt` `trashedAt` `order`	三态 + 回收站 + 排序权重
组织	`priority` `location` `subtasks[]` `source` `reminder`	来源区分手动、AI 生成、导入，用于界面标识

表 3-1 `Todo` 字段的六组划分

`isDeadline` 是这套模型里最关键的一个开关。普通待办只有截止时间点； 报名、作业、项目这类事项有起止区间。开启后表单会多出开始日期，列表卡片改显示「9-23 至 9-25」， 日历视图会把整段日期染色。之所以用一个布尔量而不是两个独立模型，是为了让筛选、排序、统计 只需要处理一种类型——代价是这个字段在每一处渲染逻辑里都要判断一次。

3.2 优先级：数字越大越重要，权重取反用于排序

优先级是 `1|2|3|4|5` 的联合类型，5 最重要（红），1 是默认（绿）。 这个方向与直觉一致：数字大 = 更要紧。但列表排序需要「5 在前」， 所以 `PRIORITY_META` 里额外给了一个取反的 `weight`（`src/lib/types.ts:70-76`）：

```
export const PRIORITY_META = {
  1: { label: "1", weight: 4, color: "#a3e635", hint: "绿（默认）" },
  3: { label: "3", weight: 2, color: "#fbbf24", hint: "黄" },
  5: { label: "5", weight: 0, color: "#f43f5e", hint: "红 · 最重要" },
};
```

代码 3-1 权重的用途是排序，不是显示（`src/lib/types.ts:70-76`，此处省略 2、4 两档）

排序发生在「智能排序」模式里：先比 `weight` 升序（等价于优先级 5→1）， 再比截止日期升序，没有截止日期的排在有日期的后面（`src/app/api/ai/chat/route.ts:62-70`）。 若直接用 `priority` 做键，就得在每处排序里写一次降序，容易漏。

3.3 历史数据收敛

早期版本用过字符串优先级（`urgent` / `high` / `medium` / `low`），也接受过中文描述。数据一旦落到用户机器上，升级时不可能要求用户重录，所以读取路径上必须有一层收敛：

```
export function coercePriority(p: any): Priority {
  if (p === null || p === undefined || p === "") return 1;
  if (typeof p === "number" && PRIORITIES.includes(p)) return p;
  const s = String(p).trim().toLowerCase();
  if (/^[1-5]$/.test(s)) return Number(s) as Priority;
  if (/(urgent|紧急|最高|最重要|红)/.test(s)) return 5;
  if (/(high|高|较快|尽快)/.test(s)) return 4;
  if (/(medium|mid|中|普通|正常)/.test(s)) return 3;
  if (/(low|低|不急|有空|以后)/.test(s)) return 2;
  return 1;
}
```

代码 3-2　任意输入收敛为 1-5（`src/lib/types.ts:79-90`）

这个函数在三个地方被调用：仓储读取时逐条迁移（`repository.ts:31-34`）、新建时兜底（`repository.ts:61`）、AI 抽取结果入库前（`ai-core.ts:155`）。同一个函数管住三条入口，就不会出现「界面上是红色、数据里是 urgent」的错位。

3.4 状态与完成时间必须联动

`done` 是布尔量，`status` 是三态字符串，两者并存看起来是冗余，实际是历史兼容：早期只有勾选完成，后来加了「进行中」，为了不破坏旧数据才保留双字段。冗余的代价是必须保证一致，规则写在仓储的更新方法里（`repository.ts:127-134`）：

- 传了 `status`：以它为准，`done` 与 `completedAt` 跟着改。
- 只传了 `done`：反推 `status` 为 `done` 或 `todo`，并同步时间。
- 两个都没传：不动状态，只更新 `updatedAt`。

这条规则只放在仓储层，不放组件。界面上的勾选、Agent 的 `set_status`、自动维护的课程流转，走的都是同一个入口，因此不会出现「勾上了但完成时间是空」的状态。



图 3-1 状态机。虚线为自动规则，30 天与 5 天两条阈值都写在 `repository.autoMaintain` 里。

3.5 子任务与来源标记

`subtasks` 是 AI 拆解的产物：一句话「准备答辩」会被拆成提纲、PPT、彩排三条。 子任务没有独立 id 生成器，新增时由调用方补齐（`src/app/api/todos/route.ts:82-87`）， 完成度只用于显示进度条，不参与主状态的判断——这是有意的简化：如果子任务全勾就自动完成主任务， 用户会失去「最后确认一遍」的机会。

`source` 只有三个值：`manual`、`ai`、`import`。 它对功能没有影响，但对信任有影响：卡片左上角的小标记让用户知道这条是模型生成的， 需要复核。 批量导入的分支会强制写 `import`（`src/app/api/todos/route.ts:80`）， Agent 创建则强制写 `ai`（`tools.ts:350`），不依赖模型自己填。

3.6 云库字段映射

本地 JSON 用驼峰，云库用下划线，这是两套命名习惯的妥协。映射写在 `cloud-repository.ts:27-74` 的 `rowToTodo` 与 `todoToRow` 里， 一一对应，没有隐式转换。表结构见 `db/schema.sql`： 21 列、两个索引（`order` 与 `due_date`），数组用 `text[]`，子任务用 `jsonb`。

有一处必须注意： `order` 是 SQL 保留字，建表与查询里都加了双引号 （`db/schema.sql:19`）。少写这对引号，建表会直接失败。

04 仓储层：本地文件与云适配

REPOSITORY · PERSISTENCE

> 这一层只回答一个问题：数据存在哪里、怎么保证读写一致。 默认实现是一个 JSON 文件，云库是可选替换项，两者接口完全相同。

4.1 一行分发

切换存储不需要改任何业务代码，判断条件只有一行（`src/lib/server/index.ts:5-9`）：

```
const useCloud = Boolean(process.env.CLOUD_ENDPOINT && process.env.CLOUD_PUBLISHABLE_KEY);
export const repository: any = useCloud ? cloudRepository : localRepo;
export const dataMode = useCloud ? "cloud" : "local";
```

代码 4-1 两个环境变量同时存在才切云（`src/lib/server/index.ts:5-9`）

用 `any` 是有意的：两套实现的返回类型一致，但内部签名略有差异（云库多一个用户维度）， 强类型会让调用方被迫感知实现细节。代价是丢失类型检查，所以两边的公开方法必须靠人工保持同步—— 第 12 章列了这 11 个方法的清单，替换存储时照着实现即可。

4.2 本地仓储的十一个方法

`repository.ts` 导出 11 个方法： `list` 、 `get` 、 `create` 、 `bulkCreate` 、 `update` 、 `restore` 、 `remove` 、 `removeMany` 、 `clear` 、 `autoMaintain` 、 `emptyTrash` 。 内部只有四个私有函数： `ensureFile` 、 `readAll` 、 `writeAll` 、 `normalize` 。

所有写操作都是「全量读 → 改内存 → 全量写」。文件不大时这是最简单的正确做法， 也让 `bulkCreate` 可以一次写盘完成 N 条插入（`repository.ts:100-111`）。 `normalize` 负责兜底：空标题变成「未命名待办」，缺省状态补 `todo` ， 新建项的 `order` 取当前最小值减一，所以新条目总在最前面。

4.3 三处必须说清的局限

文档如果不写局限，就等于把风险留给下一个人。这个仓储有三处真实缺陷，都在读代码时可以直接看到：

问题	表现	根因
写不原子	进程被强杀时可能留下截断的 JSON	直接覆盖写，没有「临时文件 + rename」（`repository.ts:43`）
解析失败静默返回空	损坏文件被读成 0 条，下一次写入即覆盖为全空	`readAll` 的 `catch` 直接 `return []`（`repository.ts:35-37`）
读改写无锁	两个请求同时改，后写的一方覆盖前一方	`writeChain` 只串行化写动作，不覆盖「读-改-写」整体（`repository.ts:14,43-44`）

表 4-1 本地仓储的已知局限

单用户本地应用里，这三个问题出现的概率不高：写入是毫秒级，用户不会同时点两次。 但 Agent 的批量工具会连续调用 `create` ，每条都触发一次全量读写， 20 条就是 20 次全文件重写。这是第 7 章提到

「`create_many` 应改用 `bulkCreate`」的原因——不是性能洁癖，而是减少窗口期。

改进方向

把 `writeAll` 改成写 `todos.json.tmp` 后 `rename`，原子性即可解决； 解析失败时把损坏文件备份为 `todos.json.bad` 再返回空数组，可避免静默清库。两处改动都在 10 行以内，之所以没做， 是因为会改变「首次启动自动建文件」的行为路径，需要重新跑一遍安装验收。

4.4 自动维护：顺序本身就是规则

`autoMaintain` 一个方法里做了三件事，每件事都带 `continue` （`repository.ts:186-232`）。顺序不能交换，原因看图就清楚：

```
// ① 逾期 30 天 → 入垃圾箱
if (!t.trashedAt && t.status !== "done" && t.dueDate && t.dueDate < cutoffTrash) {
  t.trashedAt = nowIso; trashed++; continue;
}
if (t.trashedAt) continue;
// ② 逾期 5 天 → 自动完成
if (t.status !== "done" && t.dueDate && t.dueDate < cutoffDone) {
  t.status = "done"; t.done = true; completed++; continue;
}
```

代码 4-2 两条阈值的判定顺序（`repository.ts:198-217`，省略时间戳赋值）

如果先判 5 天，一条逾期 40 天的数据会先被标成「已完成」并 `continue`， 永远走不到入箱分支，垃圾箱规则形同虚设。同理，已入箱的数据必须提前 `continue`， 否则会在垃圾箱里被标成完成，用户恢复出来看到的是一个已完成的任务。

第三条规则只对「课程」生效：根据当前时间与课时窗推算应该是 `todo`、`doing` 还是 `done`，变化时才写回（`repository.ts:219-228`）。整段逻辑最后只写一次盘， 条件是两个计数器有任意一个大于零。

4.5 云库适配器的代价

`cloud-repository.ts` 用 PostgREST 风格的条件构造器，接口签名与本地逐一对齐。 功能上等价，性能模型完全不同：本地一次全量写盘，云库是每条一次网络往返。 批量维护 N 条数据，云库要发 N 次 `update`（`cloud-repository.ts:180-218`）， 在弱网下会明显变慢。

另一个细节是客户端单例：SDK 是动态 `import` 的（`cloud-repository.ts:14`）， 本地模式不会加载它，因此安装包里虽然有这个文件，但不会产生任何网络请求。 这保证了「本地优先」不是一句口号：没有配置云库时，进程只读写磁盘。

4.6 撤销为什么要单独一个方法

`restore` 看起来可以用 `update` 代替，实际不行。 `update` 会做状态联动：传入 `status` 就把 `completedAt` 改成当前时间。 撤销要求的是「精确回到快照那一刻」，时间戳被改写就等于没恢复干净。 所以 `restore` 直接展开快照，只更新 `updatedAt`，不碰任何联动逻辑 （`repository.ts:140-155`）。

05 接口层设计

ROUTES · ERRORS · RUNTIME

> 19 个处理函数分布在 14 个路由文件里。接口数量的控制不算极简，但每个端点都有明确归属， 没有出现「一个万能接口什么都干」的情况——除了 `/api/todos` 的 POST，那里确实承担了五种写法。

5.1 按领域分组

领域	端点	职责
日程 CRUD	`/api/todos` 、 `/api/todos/[id]`	列表筛选、新增、批量、改、删、清空
Agent	`/api/agent` 、 `/agent/patrol` 、 `/agent/undo`	执行、只读巡检、撤销
模型	`/api/ai/chat` 、 `/api/ai/test`	五种文本任务、连通性测试
导入导出	`/api/import/parse` 、 `/api/export` 、 `/api/export/image`	多格式解析、三种导出、分享长图
运行支撑	`/api/auto` 、 `/api/health` 、 `/api/network` 、 `/api/qr`	自动维护、指纹探活、局域网地址、二维码

表 5-1 接口按领域分组。完整清单见附录 A。

运行支撑这一组容易被忽略，但它们决定了「装到别人电脑上能不能用」。 `/api/health` 只返回 14 行（`src/app/api/health/route.ts:6-13`）， 内容是一个固定指纹：应用名、版本、平台、架构。启动器靠它判断 3000 端口上跑的是不是自己， 而不是别的程序——这个设计在第 10 章会展开。

5.2 一个端点，五种写法

`POST /api/todos` 同时承担清空、批量删除、清空垃圾箱、批量导入、单条新增。 用 `action` 字段区分前三种，用 `items` 与 `title` 区分后两种：

```
if (body?.action === "clear") { ... }
if (body?.action === "delete" && Array.isArray(body.ids)) { ... }
if (body?.action === "empty-trash") { ... }
if (Array.isArray(body.items)) {
  // 批量导入：source 强制 import，子任务补 id
  const created = await repository.bulkCreate(inputs);
  return NextResponse.json({ todos: created, count: created.length }, { status: 201 });
}
if (typeof body.title === "string") { ... }
return NextResponse.json({ error: "缺少 title 或 items" }, { status: 400 });
```

代码 5-1 五个分支的顺序判定（`src/app/api/todos/route.ts:46-115`，省略字段映射）

这么写的收益是客户端只需要一个创建入口；代价是这个函数有 78 行，且分支顺序不能乱—— `items` 必须在 `title` 之前判，否则带 `items` 的请求若同时有 `title`， 会被当成单条创建，只插入一条。

5.3 错误码约定

状态码不是装饰，前端会根据它决定「弹提示」还是「弹配置引导」。约定如下：

状态码	使用场景	前端处理
400	参数缺失、JSON 非法、未配置 Key（带 `needConfig`）	显示文案；有 `needConfig` 时跳设置
404	日程不存在、撤销记录已过期	刷新列表
413	上传文件超过 15 MB	提示压缩或拆分
415	格式不支持（含本地模式只收 JSON 的两个 `code`）	按 `code` 给不同的安抚文案
422	文件能读但解析失败	提示换一份文件
501	找不到可用浏览器，无法导出图片	提示安装 Edge/Chrome
502	上游模型接口异常	透出服务商原文与建议

表 5-2 状态码与前端行为的对应关系

415 有两个业务码：`LOCAL_ONLY_JSON` 表示「没接 AI，只能导 JSON」，`LOCAL_JSON_INVALID` 表示「是 JSON 但不符合本应用格式」。分成两个码，是因为前者要引导用户去配 Key，后者要引导用户换文件，文案完全不同（`src/app/api/import/parse/route.ts:356-375`）。

5.4 三个运行配置

配置	位置	作用
`dynamic = "force-dynamic"`	几乎所有 `route.ts` 顶部	禁止构建期静态化，保证每次请求都读最新文件
`maxDuration = 120`	`agent/route.ts:10`	给 Agent 留出比 90 秒墙钟更宽的运行上限
`runtime = "nodejs"`	`import/parse/route.ts:17`	解析要用 `Buffer`、`xlsx`、`mammoth`，不能跑在 Edge Runtime

表 5-3 路由级运行配置

`force-dynamic` 是必须的。Next.js 默认会把没有动态参数的 GET 在构建期预渲染，对 `/api/todos` 这种读本地文件的接口来说，预渲染意味着构建机上的空数据被固化进产物。加上这行之后，每次请求都会真正走一遍 `repository.list()`。

5.5 为什么模型出错也返回 200

`POST /api/agent` 在模型不可用时返回的是 HTTP 200，响应体里 `stopped_reason` 为 `model_error`，并带 `error` 字段（`src/lib/server/agent/runtime.ts:327-340`）。这不是疏忽。

对界面来说，「跑完了但没成功」和「请求失败」需要不同的展示：前者要显示完整轨迹与失败原因，让用户看到 Agent 做了什么；后者只能弹一个 toast。如果统一用 5xx，前端拿不到结构化的 `steps`，轨迹面板会是空的，用户不知道发生了什么。这条约定写在路由注释里：**Agent 的运行结果永远是一个可渲染的对象，而不是一个错误。**

06 AI 能力层

MODEL CALLS · EXTRACTION

> 这一层不实现 Agent，只做三件事：把请求发出去、把错误翻译成人话、把模型返回的脏数据洗干净。Agent 的规划逻辑在下一章，两者共用这里的调用与清洗能力。

6.1 统一调用层

所有请求都走 OpenAI 兼容协议，`POST {baseUrl}/chat/completions`。 配置解析有明确的优先级：请求里带了 Key 就用请求的，否则读环境变量 （`src/lib/server/ai-core.ts:21-38`）。这解决了两个场景的冲突： 用户在自己电脑上填的 Key 存在浏览器里，服务端不应该也没有必要持久化； 而部署到服务器时又需要环境变量兜底。

```
if (reqKey) {
  return { baseUrl: cfg.baseUrl || "https://api.openai.com/v1",
           apiKey: reqKey, model: cfg.model || "gpt-4o-mini", source: "request" };
}
return { baseUrl: process.env.OPENAI_BASE_URL || ..., apiKey: envKey,
         model: process.env.OPENAI_MODEL || "gpt-4o-mini", source: "env" };
```

代码 6-1 请求级配置优先于环境变量（`src/lib/server/ai-core.ts:21-38`，省略 trim）

温度分两档：抽取类任务 0.5（`ai-core.ts:118`），Agent 规划 0.2（`runtime.ts:118`）。 抽取需要一点语言灵活度来理解口语，规划需要稳定复现，所以压得更低。 需要 JSON 时额外传 `response_format: { type: "json_object" }`， 但代码没有依赖它——模型不遵守时还有正则兜底，见 6.4。

6.2 两套抽取提示词

文本抽取与图片识别用的是两份不同的系统提示词，而不是一份加上「如果是图片」的分支。 原因是两类输入的失败模式完全不同：文本可能包含大段无关内容，需要判断「哪些句子是任务」； 图片可能是课表、白板、聊天截图，需要先做版面理解，再按行拆条。

约束	文本提示词	图片提示词
输出格式	固定 JSON schema，只输出 JSON	同一 schema，强调「图中所有」
日期换算	以今天为基准换算「周三」「下周一」	无法确定年份时按今年处理
长时段	跨多日时 `isDeadline=true`，填起止	只写截止日的也要置 `true`
专项规则	标签用 1-3 个中文短词	课表按「科目 + 时间 + 地点」逐条列
输入截断	正文 12,000 字符	整图 base64，不截断

表 6-1 两份提示词的差异（`ai-core.ts:53-72`）

两份提示词里都有 `${todayStr()}` 插值，没有使用反引号包裹示例 JSON。 这是个踩过的坑：模板字符串里出现裸反引号会提前结束字符串，构建直接失败（第 11 章有记录）。

6.3 把错误翻译成人话

模型服务商的错误信息对普通用户没有意义。这一层把 9 个常见状态码映射成「发生了什么 + 该做什么」，并保留服务商原文（`ai-core.ts:75-97`）：

码	文案
401	API Key 无效或已过期，请到服务商控制台重新生成后粘贴。
402	账户余额不足（欠费），请到服务商控制台充值后再试。
403	没有访问权限：可能是地区限制，或该 Key 未开通这个模型。
404	接口地址或模型名不存在，请检查 Base URL 与模型名称。
429	请求太频繁或额度已用尽，请稍后再试。
5xx	服务商内部错误 / 网关异常 / 繁忙，请稍后重试。

表 6-2 错误中文化映射（节选）

区分 401 与 402 特别重要。调试期遇到过两次「接入后无法对话」，一次是 Key 写错（401），一次是账户欠费（402），用户看到的都是「请求失败」。分开之后，用户能直接知道是去重新生成 Key 还是去充值。

6.4 四道清洗

模型返回的 JSON 不可信，入库前必须过一遍 `toTodoInputs`（`ai-core.ts:147-168`）。四道处理按顺序执行：

1. **容错解析**：先直接 `JSON.parse`，失败则用 `/\{[\s\S]*\}/` 抓第一个完整对象。模型常在 JSON 前后加解释文字或 ```json 围栏。
2. **字段裁剪**：标题 200 字、备注 500 字、地点 100 字、标签最多 5 个。防止一条超长文本把卡片撑破。
3. **格式校验**：日期必须匹配 `yyyy-mm-dd`，时间必须匹配 `H:mm` 或 `HH:mm`（补零到 5 位），不合格的直接置 null，而不是丢弃整条。
4. **业务规则**：识别为课程的，优先级强制为 1 并补「课程」标签（`ai-core.ts:151-156`）。这条规则写在服务端，前端与 Agent 都绕不过去。

第三点的选择值得说明：校验失败时置 null 而不是抛错，是为了保住标题与备注—— 用户更在意「这条有没有进来」，而不是「日期对不对」。日期错了可以手动补，整条丢了要重新导入。

6.5 没有 Key 时的兜底

不能假设每个用户都有 API Key。没有 Key 时，系统仍然可用，只是精度下降：

- **导入**：只接受本应用导出的 JSON，或手动逐行输入（第 9 章）。
- **智能添加**：`localExtractTodos` 按换行与标点分句，用关键词判断优先级（「紧急/务必」→5，「今天/尽快」→4，「有空/以后」→2），识别 `#标签` 与「今天」。
- **排序/总结/拆解**：走本地规则。排序按「优先级 → 截止日期」，总结直接统计各状态数量。
- **对话**：明确回复「还没配置大模型」，并给出配置路径，不假装能回答。

这条设计的判断是：**宁可功能少，不要功能假**。用规则硬凑一段「智能总结」， 用户第一次觉得神奇，第二次就发现它什么都没说。明确告诉用户缺什么，比编一个答案更有用。

6.6 Key 为什么存在浏览器

配置存在 `localStorage` 的 `scheduleApp.ai.v1` 键下（`src/lib/ai-config.ts:64`）， 每个请求随 body 下发。这样做的好处是：安装包里不含任何密钥，换一台电脑要重新填， 也就不存在「打包时把别人的 Key 带出去」的风险。代价是用户换浏览器要重填一次， 以及任何能执行 JS 的页面都能读到它——这在本地单机场景下可以接受，但如果将来做多用户部署， 这套方案必须换成服务端密钥托管。

6.7 上下文是怎么被裁掉的

多轮工具调用会让消息数组迅速变长：每轮一次 assistant（含 tool_calls）加最多四条 tool 结果， 六轮下来很容易超过模型窗口。这里的做法是「按字符估算 + 优先丢工具结果」， 而不是引入分词器——估算精度足够，代价是没有依赖。

`trimMessages` 先把每条消息折算成「内容长度 + 40」并求和，与 `MAX_PROMPT_CHARS`（24,000）比较；超了就从最后一条往前回填， 遇到 `role: "tool"` 且丢弃后仍不低阈值的，直接跳过（`runtime.ts:247-261`）。最后强制保留 `messages[0]`，也就是系统提示词——它不能被裁掉，否则规则全失效。

已知风险

只丢 `tool` 消息、保留对应的 `assistant.tool_calls`，在严格实现的服务商那里 会报「tool_call_id 没有对应的响应」。当前模型与主流网关都容忍这种情况，所以没有触发； 如果要接更严格的接口，正确做法是连同 `assistant` 那一轮的 `tool_calls` 一起丢， 保证配对完整。

单条工具结果本身也有上限：回灌前统一截断到 4,000 字符（`runtime.ts:455-463`）。 这意味着 `list_todos` 一次返回 50 条时，后面的条目会被截掉—— 所以工具层默认只返回 20 条，并在 `summary` 里写明「共 N 条，返回前 M 条」， 让模型知道还有更多，必要时再缩小条件重查。

07 Agent 设计

BOUNDED LOOP · TOOLS · SAFETY

> 这一章是全文最长的一章，因为 Agent 是这个作品里唯一「行为不可完全预测」的部分。 不可预测的东西要能用，只能靠边界：模型负责选工具，控制器负责所有约束。

7.1 三条设计原则

`tools.ts` 开头写了三条原则（`src/lib/server/agent/tools.ts:1-5`）， 它们决定了后面所有细节：

1. **工具只做最小必要动作**，删除类操作永远只提议、不执行。模型没有「删除」这个工具可用。
2. **参数错误返回可读错误**，让模型自己修正重试，而不是抛异常打断循环。
3. **返回值带 summary**，控制回灌给模型的上下文体积，4000 字符截断。

第一条是安全底线。如果给模型一个 `delete_todo`，它迟早会在某次误解指令时调用； 一旦调用，用户的数据就没了。所以工具集里只有 `propose_delete`， 它做的事是把 id 和标题写进 `ctx.pending`，等用户在界面上点确认。

7.2 控制器与预算

六个常量定义了这个 Agent 的能力上限（`runtime.ts:8-13`）：

常量	值	作用
`MAX_STEPS`	6	最多六轮「模型思考 + 执行工具」
`MAX_WALL_MS`	90,000	整轮墙钟上限，含所有模型调用与工具执行
`CALL_TIMEOUT_MS`	30,000	单次模型调用的 `AbortController` 超时
`MAX_TOKENS`	30,000	累计 Token 预算，按服务商返回的 usage 累加
`MAX_PROMPT_CHARS`	24,000	上下文裁剪阈值，超了就丢最早的 tool 消息
`MAX_TOOL_CALLS_PER_STEP`	4	一步最多执行四个工具调用，多余的丢弃

表 7-1 运行时硬约束（`src/lib/server/agent/runtime.ts:8-13`）

主循环是一个带标签的 `for`，标签是为了让熔断能从两层嵌套里直接跳出（`runtime.ts:355`）：

```
outer: for (let step = 0; step < MAX_STEPS; step++) {
  if (Date.now() - startedAt > MAX_WALL_MS) { stopped = "timeout"; break; }
  if (usage.total_tokens >= MAX_TOKENS) { stopped = "budget"; break; }
  const reply = await callModel(ai, trimMessages(messages), planningMode === "tools", toolsForModel);
  if (!reply.tool_calls.length) { answer = reply.content; stopped = "final"; break; }
  for (const call of reply.tool_calls.slice(0, MAX_TOOL_CALLS_PER_STEP)) {
    const result = await executeTool(name, args, { pending, mutations });
    messages.push({ role: "tool", content: JSON.stringify(result).slice(0, 4000) });
  }
}
```

代码 7-1 主循环骨架（`runtime.ts:355-479`，省略错误处理与熔断）

每一步都先查时间与预算，再去调模型。顺序很重要：如果把模型调用放在检查之前， 最后一步可能已经超时却还在发请求，用户要多等 30 秒才看到「超时」。

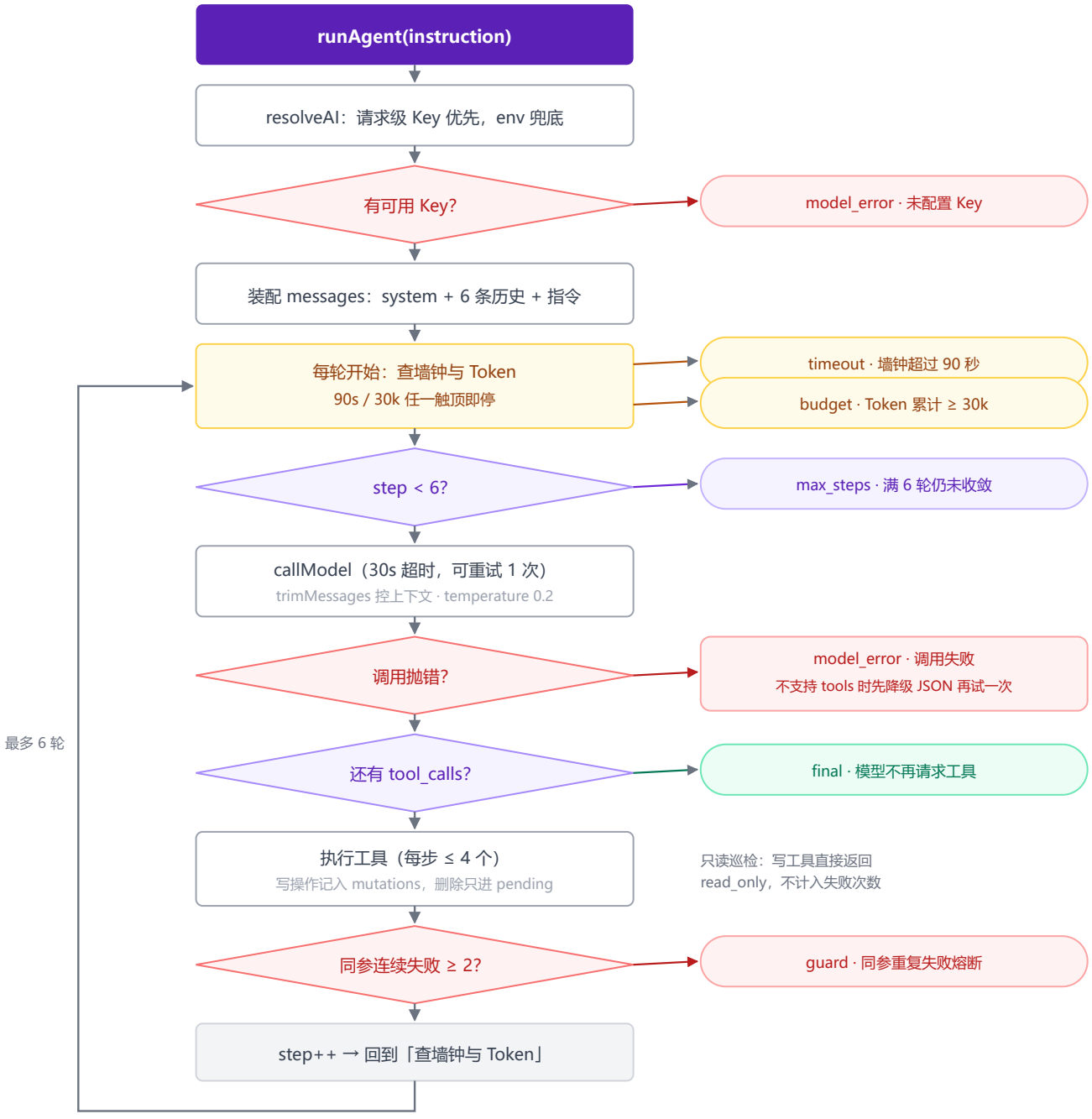


图 7-1 有界循环与六个出口。右列每个出口都对应 `stopped_reason` 的一个取值，界面据此显示不同的结束语。

7.3 九个工具

工具	类型	作用	关键约束
`list_todos`	只读	按状态、标签、关键词、截止日筛选	limit 1-50，隐式排除垃圾箱
`create_todo`	写	新建一条	title 必填；来源强制 `ai`
`update_todo`	写	局部更新任意可改字段	id 必填；开始日晚于截止日被拒
`set_status`	写	改状态并联动完成时间	只能是 todo / doing / done
`stats_overview`	只读	总量、各状态、逾期、未来 N 天分布	days 1-14
`propose_delete`	提议	生成待确认清单，不删除	ids 1-20；理由 ≥ 4 字
`find_conflicts`	只读	同日时间窗重叠检测	缺 endTime 按 +120 分钟算；最多 10 组
`reschedule_many`	写	批量改到某天或整体顺延	ids 1-30；`date` 与 `shiftDays` 二选一
`create_many`	写	一次建多条	1-20 条；部分成功也返回成功项

表 7-2 工具清单（schema 在 `tools.ts:72-251`，执行器在 `tools.ts:288-621`）

后三个工具是后期补的，动机都来自实际使用：用户说「这周三和周四的时间冲突了吗」，没有 `find_conflicts` 时模型只能把两条都读出来再自己算，经常算错；说「把这几门课整体往后推一周」，没有 `reschedule_many` 就要循环调用 `update_todo`，六步上限根本不够。

7.4 参数校验：给模型纠错的机会

工具出错时不抛异常，而是返回 `{ ok: false, error, summary }`。模型能从 `error` 里读到具体原因，下一轮改参数重试。以日期为例：

```
const dueDate = validateDate(args.dueDate);
if (args.dueDate !== undefined && dueDate === undefined)
  return { ok: false, error: "dueDate 必须是 yyyy-mm-dd", summary: "创建失败：日期格式错误" };
```

代码 7-2 只在「显式提供了非法值」时报错（`tools.ts:328-330`）

`args.dueDate !== undefined` 这个前置判断是踩坑换来的。早期写法是 `if (!dueDate) return 失败`，结果模型创建一个只有标题的日程时，因为没传 `dueTime`，被判定为「时间格式错误」，创建零条、失败两条。可选字段的正确语义是：**不传就是不要，传了才校验**。同样的模式在 `create_todo`、`update_todo`、`create_many` 三处都出现过。

7.5 六个停止条件

取值	触发	用户看到什么
`final`	模型不再请求工具，给出自然语言答复	正常结果
`max_steps`	跑满 6 轮仍有工具调用	「已达最大步数……最后一步是 X」，并提示拆分
`timeout`	墙钟超过 90 秒	「超时，可缩小范围后重试」
`budget`	Token 累计达到 30k	「预算用完，建议拆小」
`guard`	同一工具同一参数连续失败两次	指出是哪个工具、什么原因，建议检查 id/日期
`model_error`	未配置 Key，或调用彻底失败	配置引导或服务商原文

表 7-3 `stopped_reason` 的六个取值（`runtime.ts:35,355-489`）

熔断（`guard`）值得单独说。模型有一种固定失败模式：参数写错导致工具报错， 它以为「再试一次就好」，于是原样再发一遍，直到步数耗尽。两次之后强制停止， 既省额度，也让用户更快看到原因：

```
if (!result.ok) {
  const sig = `${name}:${JSON.stringify(args)}`;
  const n = (failureCount.get(sig) || 0) + 1;
  failureCount.set(sig, n);
  if (n >= 2) { stopped = "guard"; answer = `检测到重复失败（${name} 连续 ${n} 次…）`; break outer; }
} else {
  failureCount.delete(sig);
}
```

代码 7-3 按「工具名 + 参数」签名计数的熔断（`runtime.ts:466-478`）

7.6 JSON 降级

不是所有兼容接口都支持 function calling。有些网关会直接返回 400， 有些模型会忽略 `tools` 字段只回一段文字。遇到前者时需要降级， 但降级不能对所有 400 都触发——模型名写错也是 400，那种情况降级只会掩盖错误。 判断条件必须是「状态码 + 关键字」同时命中（`runtime.ts:197-201`）：

```
function isToolsUnsupported(msg: string): boolean {
  const code = /返回\s*(400|404|422)/.test(msg);
  const keyword = /tool|function|unsupported|not support/i.test(msg);
  return code && keyword;
}
```

代码 7-4 降级触发条件（`runtime.ts:197-201`）

降级后改用纯文本协议：模型每一步只输出一个 JSON 对象， `{"action":"tool","tool":"...","args":{...}}` 或 `{"action":"final","answer":"..."}`。 控制器把它翻译成统一的 `tool_calls` 结构，后面的执行路径完全复用。 解析器会先剥掉 ```json 围栏，再取第一个 `{` 到最后一个 `}`， 这样模型多说一句「好的，我来处理」也不会失败（`runtime.ts:223-244`）。

这条链路有专门的验证方式：假模型服务提供 `reject_tools` 模式， 只要请求带 `tools` 就返回 400 与 `tools is not supported`， 正好命中降级条件（`scripts/mock-openai-server.mjs:32-38`）。

7.7 删除为什么必须由人确认

模型永远拿不到删除工具，只能提议。这条规则在三个层次上落地，少了任何一层都不成立：

层次	做法	证据
提示词	明确「删除只能用 propose_delete，禁止声称已删除」	`runtime.ts:76,78,84`
工具集	不存在 delete 类工具；`propose_delete` 只写 `ctx.pending`	`tools.ts:10-19,467`
界面	确认卡片列出标题，用户点「移入垃圾箱」才调用软删除	`AgentPanel.tsx:186-199,428-473`

表 7-4 人在环的三重落地

注意第三层用的是软删除：`trashedAt` 置为当前时间，记录仍在文件里（`store.ts:134-139`）。 即使用户点错了，还能在垃圾箱里恢复。这是「不可逆操作要留后路」的直接体现， 也是为什么 Agent 的删除不需要额外的「二次确认弹窗」——确认卡本身已经是二次确认。

7.8 撤销栈

有写操作就可能写错。撤销的实现分三段：记录、落盘、回放。 记录只存必要字段（`tools.ts:29-49`）：

```
export interface Mutation { type: "create" | "update"; id: string; before?: Partial<Todo>; }

const UNDO_FIELDS = ["title","notes","priority","status","tags","dueDate",
  "dueTime","endTime","startDate","isDeadline","location","done","completedAt"];
```

代码 7-5 还原点结构（`tools.ts:29-49`，数组已折行）

白名单而不是整条快照，是为了让日志文件小、可读，也避免把 `id`、`order` 这类不该改的字段写进去。回放时按逆序执行（`undo/route.ts:27-38`）：

```
for (const m of [...entry.mutations].reverse()) {
  if (m.type === "create") await repository.remove(m.id);
  else if (m.type === "update" && m.before) await repository.restore(m.id, m.before);
}
```

代码 7-6 逆序回滚（`src/app/api/agent/undo/route.ts:27-38`，省略计数）

必须逆序。假设一次运行先创建 A、再把 B 改名、最后又把 A 改名：正序回滚会先删 A， 后面两条针对 A 的恢复就找不到目标。日志只保留最近 20 条（`log.ts:7-8`）， 撤销成功后立即删除该条记录，重复撤销返回 404（`undo/route.ts:19-22`）。

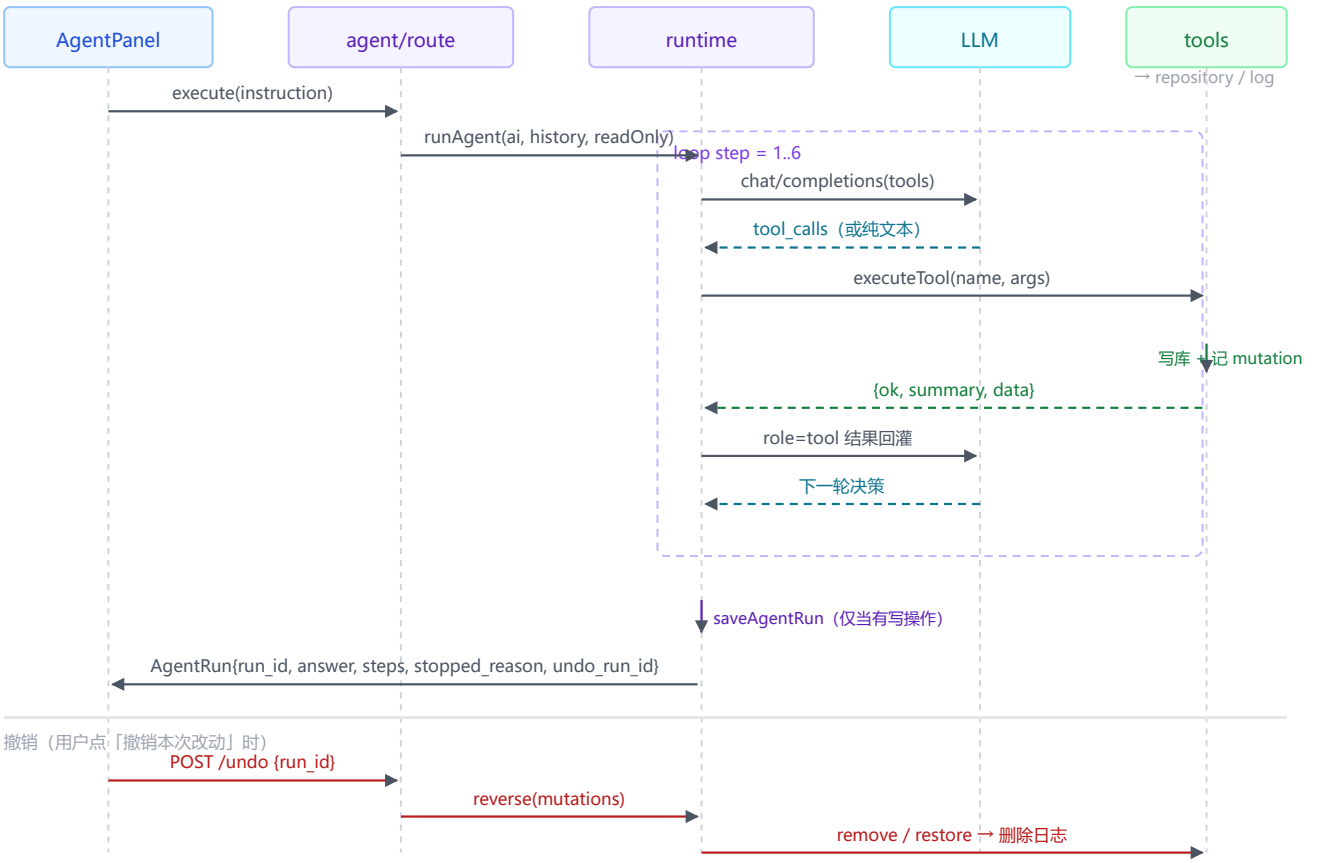


图 7-2 一次运行的时序。实线是请求，虚线是响应；底部一条独立泳道是撤销，它只依赖落盘的 `run_id`，与这次运行隔多久无关（20 条以内）。

7.9 只读巡检与越权检测

「今日简报」是定时任务，不应该有机会改数据。实现上做两件事：只暴露三个只读工具， 并在执行前再拦一次写工具（双保险，防模型硬编一个工具名）。

更难处理的是模型的「嘴上说改完了」。只读模式下，写工具会返回 `{ ok: false, summary: "只读巡检模式禁止修改数据" }`，但模型完全可能忽略 `ok`，回一句「已为你调整完毕」。如果直接采用这段文本，用户会以为数据变了。 所以巡检路由在采用结果前做了两道检查（`patrol/route.ts:105-115`）：

```
const run = await runAgent({ instruction, ai, readOnly: true });
const blockedWrite = run.steps.some((s) => s.summary.includes("只读巡检模式"));
if (run.answer && run.stopped_reason === "final" && !blockedWrite) {
  summary = run.answer; engine = "agent";
} else {
  summary = localBrief(todos);   // 回退到本地统计
}
```

代码 7-7 有越权尝试就丢弃模型结果（`src/app/api/agent/patrol/route.ts:105-115`）

这是一种「不信任模型自述」的通用模式：判断依据是执行轨迹，不是模型的结论。 同样的思路也用在 `ok:false` 的处理上——提示词要求模型如实说明失败原因， 但真正的保障是轨迹里能看到失败记录。

7.10 评测与假模型

改提示词、加工具之后，怎么知道没把原来的能力弄坏？仓库里有两个配套工具。

`scripts/eval-agent.mjs` 是 14 条评测用例，分正常、边界、对抗三类（`scripts/eval-agent.mjs:1-2`）。每条用例给出输入和断言函数， 断言检查的是轨迹而不是措辞：例如「只读概览」要求 `stopped_reason === "final"` 且至少调用过一个工具；「课程查询」要求调用了 `list_todos`。 这样即使模型换了措辞，测试也不会误报。

`scripts/mock-openai-server.mjs` 是一个本地假模型服务，兼容 OpenAI 协议， 提供 8 种模式：`create`（正常创建）、`reject_tools`（触发 JSON 降级）、 `bad_loop`（触发熔断）、`batch`（批量创建与撤销）、`extract` / `conflict` / `markdown` / `read`。 没有 API Key、没有外网时，也能把真实 HTTP 链路、工具执行、撤销、排版渲染全部跑一遍。

7.11 一次运行的代价

Token 用量不是估算的，而是累加服务商每次返回的 `usage`（`runtime.ts:394-396`）。 三个字段都记：`prompt_tokens`、`completion_tokens`、`total_tokens`， 预算判断只看总数。正常任务两到四步就收敛，六步是给「多条件查询 + 批量修改」留的余量。

真正的成本控制在工具层，而不是模型层。三处设计都在减少回灌体积： `list_todos` 默认只返回 20 条并附总数； `compact()` 只保留渲染必需的字段， 丢掉 `createdAt` 这类模型用不到的时间戳；工具结果统一截断 4,000 字符。 同样的数据，全量返回与精简返回的 prompt 体积能差三倍。

延时方面有一个明确的取舍：**不用流式**。Agent 需要拿到完整的 `tool_calls` 才能决定下一步，流式输出对规划没有帮助； 而界面上的等待感用另一种方式解决——执行期间显示计时器与当前步数（`AgentPanel.tsx:145-157`），用户能看到「它在动」， 比逐字输出更能缓解焦虑。

重试策略也直接影响等待时间。只有网络错误、超时、429 与 5xx 会重试一次， 退避时间是 `400 × 尝试次数 + 随机抖动`（`runtime.ts:182-191`）。 401 与 402 属于配置问题，重试没有意义，直接失败并把原因翻译成中文。

最坏情况下，墙钟会先于步数触顶：单次调用 30 秒超时、最多两次尝试，两轮之后就已经接近 90 秒， 所以用户感受到的上限是「一分半」，而不是「六轮 × 一分钟」。

结论

Agent 的可控性不来自提示词写得多严，而来自控制器的四重上限、六个明确出口、 工具层的只读约束与人在环确认。提示词只负责让模型「愿意配合」，真正的边界在代码里。

08 前端架构与富文本排版

STATE · VIEWS · RICHTEXT

> 界面部分有两个真正的难题：一是五个视图共用一套筛选条件却不互相干扰， 二是模型输出的文字层级混乱——它会把标题、列表、结论混在一段里。后者最终用一个 231 行的自研渲染器解决。

8.1 一个 store，二十九个动作

全局状态放在一个 Zustand store 里，没有 Provider，也没有按模块拆分。理由是这个应用的状态耦合度很高： 批量选择模式要读当前筛选结果，抽屉开关要读当前编辑对象，导入完成后要刷新列表并关闭弹窗。 拆成多个 store 之后，跨 store 的联动会变成一串订阅。

29 个动作用途分三类：视图与筛选（`setView`、`setFilter`）、 弹层开关（`openForm` / `closeForm` 等 10 个）、 数据操作（`load`、`create`、`update`、`remove`、 `toggle`、`bulk`、`reorder`、`trashMany`、`restore`、`emptyTrash`、`clearAll`、`runAuto`）。 数据操作一律走 `api-client`，成功后同步本地数组，不做乐观更新—— 本地文件写入是毫秒级，为了这点延迟引入回滚逻辑不划算。

派生数据用选择器算，不进 store： `selectFiltered` 根据状态、优先级、标签、关键词 过滤并排序（`src/lib/store.ts:198-215`）， `allTags` 汇总去重后的标签。 这样筛选条件变化时不需要手动触发重算，组件订阅选择器即可。

8.2 五个视图共用一套筛选

列表、日历、看板、统计、垃圾箱五个视图读的是同一个 `selectFiltered` 结果。 好处是切换视图时筛选条件不丢，用户在列表里筛出「本周 + 高优先级」，切到日历看到的还是这批。 代价是统计视图也必须接受筛选结果，而不是永远统计全量——这一点在界面上用标题文案做了提示， 避免用户误以为数据丢了。

视图	实现要点	取舍
列表	按截止日期分组，逾期置顶并用红色标注	不做虚拟滚动，数据量在千条内
日历	月网格，长时段事项跨格染色，点击格子新建	只渲染当月，不预加载邻月
看板	三列对应三态，拖拽改状态	拖拽只在桌面端启用
统计	状态分布、优先级分布、近七日完成趋势	用手写条形图，不引图表库
垃圾箱	单独查询 `?trash=1`，支持恢复与清空	与主列表完全隔离

表 8-1 五个视图的实现取舍（`src/components/views.tsx`）

8.3 三个交互细节

搜索不是即时过滤。输入框有 260ms 防抖，并且必须按回车或点按钮才真正提交 （`AppShell.tsx:75-90`）。原因是搜索会触发服务端查询，逐字发请求在本地也会造成明显卡顿。 早期版本是即时搜索，用户反馈「打字时列表一直在跳」，改成显式提交后稳定了。

下拉框是自己画的。原生 `<select>` 的展开面板由操作系统渲染， 在暗色主题下会露出一块白底，文字对比度也不受 CSS 控制。于是写了一个 153 行的 listbox （`src/components/ui/Select.tsx`），支持键盘上下选择、回车确认、Esc 关闭、点击外部收起。 这是「不引组件库」的直接代价，但只写一次，全站复用。

批量模式有独立的确认路径。进入批量模式后，卡片点击变成勾选， 底部浮出操作条；删除会先显示数量再执行。Agent 的删除确认走的是同一个 `trashMany`，因此两条路径的软删除语义完全一致。

8.4 RichText：为什么自己写

模型输出的是 Markdown 片段，但只需要其中很少的语法：标题、有序/无序列表、引用、分隔线、 加粗与行内代码。引入 `react-markdown` 会带来解析器与插件链， 体积增加几十 KB，而且默认样式与本项目的字号体系冲突，仍要重写一遍。 自己写 231 行，换来的是每个层级的字号、行高、间距都能精确控制。

解析器只做行级判断，不处理嵌套（`RichText.tsx:135-201`）：

```
const h = line.match(/^(#{1,4})\s+(.+)$/);            // 标题 1-4 级
const quote = line.match(/^>\s?(.*)$/);               // 引用
const bullet = line.match(/^[-*•·]\s+(.+)$/);          // 无序
const ordered = line.match(/^(?:\d+[.、)]|（\d+）|\(\d+\))\s*(.+)$/); // 有序
```

代码 8-1 四种行级模式（`src/components/RichText.tsx:162-186`）

有序列表的正则特意兼容了四种写法：`1.`、`1、`、`1)`、`（1）`。 模型不会固定用哪一种，实测同一个模型在两次回答里就会混用。

8.5 字号与层级

排版的核心不是「好看」，而是「一眼能分出层级」。整套体系只有四个字号档， 靠字重、颜色、前缀装饰区分层级，避免字号过多导致节奏混乱：

元素	常规	紧凑	视觉处理
一级标题	18px / 600	16px	底部 1px 分隔线，白色
二级标题	15px / 600	14px	左侧 4px 紫色竖条
三级标题	14px / 600	13px	无装饰，颜色降到 90% 白
正文	13.5px / 1.8	12.5px / 1.7	80% 白，段间距 10px

表 8-2 RichText 的字号体系（`src/components/RichText.tsx:24-27`）

列表的视觉重量也做了区分：无序列表用 1.5px 紫色圆点，有序列表用 18px 圆形编号徽章。 编号徽章比圆点重得多，因此有序列表在长回答里会自然形成「步骤感」， 这正是模型输出操作步骤时需要的语气。

8.6 把「结论」渲染成卡片

纯引用块在长文里容易淹没。这里加了一条规则：如果引用内容以「结论 / 总结 / 建议 / 下一步 / 风险」 开头并跟冒号，就渲染成紫色高亮卡片，前缀变成一个小标签（`RichText.tsx:100-116`）。

```
const m = (b.text || "").match(/^(结论|总结|建议|下一步|风险)[：:]\s*(.*)$/);
if (m) return <div className="rounded-xl border border-violet/25 bg-violet/10 …">
  <span className="… bg-violet/25 text-violet">{m[1]}</span>{m[2]}</div>;
```

代码 8-2 结论卡片的识别（`src/components/RichText.tsx:101-115`，类名已简化）

这条规则要和提示词配合才成立。系统提示词里明确要求「末尾用 > 建议：… 给出可执行建议」（`runtime.ts:82`），模型照做，渲染器就有东西可抓。 如果只改渲染器不改提示词，卡片永远不会出现；只改提示词不改渲染器，用户看到的是一段灰字引用。

8.7 暗色主题的三个具体问题

主题不是「把背景调黑」这么简单。暗色界面有三个反复出现的坑，这个项目各自用了不同的解法。

第一个是原生控件的白底。除了下拉框，日期选择器、文件选择按钮在暗色下同样会露出白块。 下拉框用自绘解决；日期输入改用文本输入加格式校验，避免调起系统面板； 文件选择则完全隐藏原生输入，用自定义按钮触发。

第二个是紫色的用法。 `#8b5cf6` 在深色底上对比度不足以承载正文， 因此只出现在三种位置：标题前的竖条、列表圆点、按钮底色。 正文用 `white/80`，次要信息用 `white/60`，标题才用纯白—— 三级明度差让层级在没有字号变化的地方也能被感知。

第三个是背景模糊的开销。 `backdrop-filter` 在低端集成显卡上会明显掉帧。 因此它只用在抽屉、弹窗、统计卡片这三类「数量有限、面积可控」的元素上， 列表项一律用纯色半透明，不用模糊。

颜色语义也固定下来：优先级 1 到 5 对应绿、蓝、黄、紫、红（`types.ts:71-75`）， 逾期统一用红色文字加左侧竖条，不再引入第六种颜色。语义颜色一旦超过五种， 用户就需要查图例，等于没有语义。

取舍

自研渲染器不支持表格、代码块、脚注、图片。这是有意的：Agent 的回答里出现表格通常意味着它在 罗列数据，而这种场景应该直接给结构化卡片；出现代码块则说明它在解释实现，超出了日程助手的职责。

09 导入与导出

IMPORT · EXPORT · SNAPSHOT

> 导入是用户第一次把「自己的东西」交给这个应用，也是最容易劝退的环节： 文件格式千奇百怪，用户不知道什么叫 JSON。所以这里做了双模式——有 AI 时什么都吃，没 AI 时明确只吃一种。

9.1 双模式判定树

导入界面默认「AI 识别」，检测到没有配置 Key 时自动切到「本地导入」并锁死可选格式。 服务端的判定更严格，因为它不能信任前端：

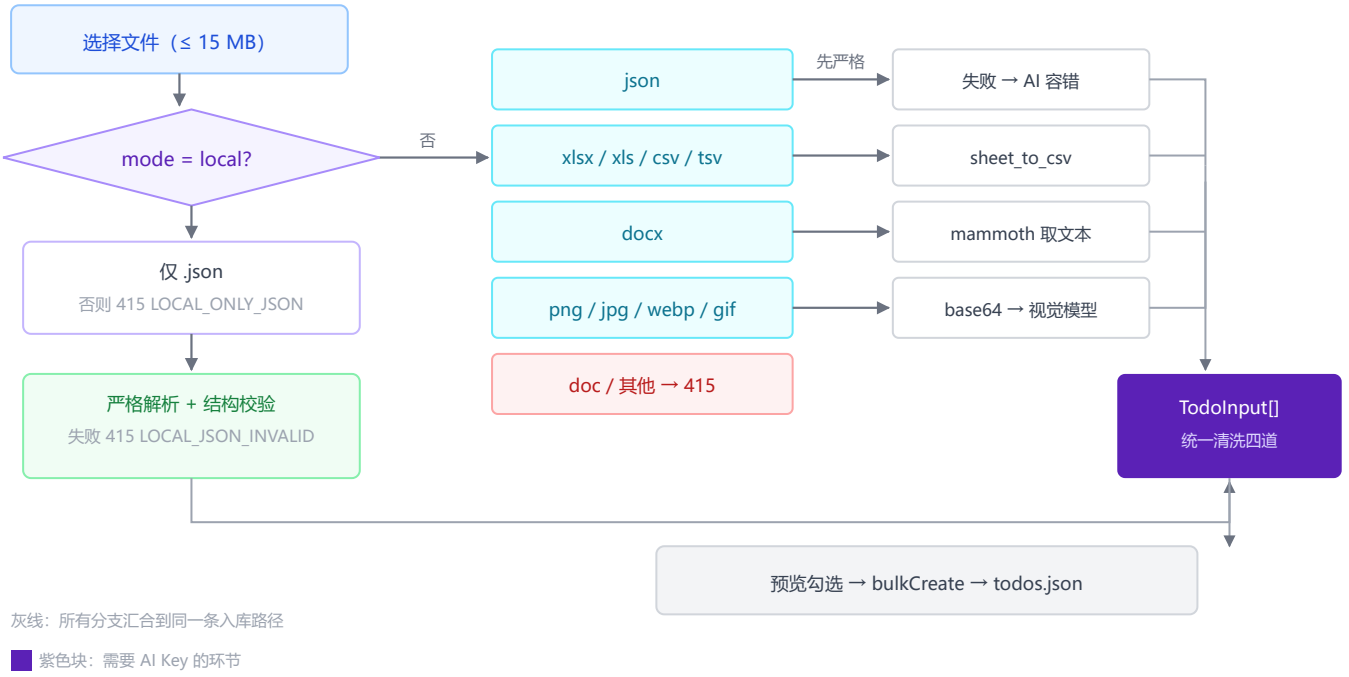


图 9-1 导入判定树。本地模式只走最左一条竖直路径，其余分支都需要 Key。

服务端对 `mode` 的读取放在函数顶层而不是 `try` 里（`import/parse/route.ts:313`）。这不是风格问题：如果声明在 `try` 内部， 后面的 `if (mode === "local")` 会拿不到值——这个作用域错误让构建失败过一次。

9.2 表格：先猜表头，再猜列

用户导出的 Excel 不会遵守任何约定：表头可能在第三行，可能叫「事项」也可能叫「任务」。 处理分两步。第一步在前 5 行里找出「命中别名最多」的一行当表头（`import/parse/route.ts:200-219`）：

```
const HEADER_ALIASES = {
  title:    ["title","标题","事项","任务","待办","名称","日程","内容"],
  dueDate:  ["date","日期","截止","截止日期","deadline"],
  priority: ["priority","优先级","重要度","级别"],
  tags:     ["tags","标签","分类","类别"],
};
```

代码 9-1 表头别名（`src/app/api/import/parse/route.ts:160-168`，节选三组）

第二步是 AI 模式下的保底做法：整张表转成 CSV 文本（每个工作表加一行 `# 工作表：名称`），截断到 20,000 字符后交给模型（`import/parse/route.ts:179-190`）。这样即使表头完全无法识别（比如整列都是自然语言描述），模型也能读懂。

为什么两种方式都保留？因为纯规则解析是确定性的、免费的、快的，但它只认表头；AI 解析能理解任意排版，但要花额度、可能出错。能用规则时用规则，规则失效时再交给模型。

9.3 docx 的三级降级

Word 文档有两种可能：一种是从本应用导出的（内部其实是 JSON 包在文档里），另一种是用户自己写的。处理顺序是「先当结构化数据，再当纯文本」：

1. 用 `mammoth.extractRawText` 取出全文（`import/parse/route.ts:272`）。
2. 尝试把全文当 JSON 解析，成功则按本应用格式直接映射（`:278-286`）。
3. 失败则交给模型抽取；模型也不可用时，退回本地启发式分句（`:289-296`）。

旧版 `.doc` 明确返回 415 并提示「另存为 .docx」（`import/parse/route.ts:400-405`），而不是静默失败。二进制格式差异太大，强行解析只会得到乱码，不如直接说清楚。

9.4 图片：一次调用，不切图

截图识别走视觉模型，图片整个转成 base64 data URL 后随消息发出（`import/parse/route.ts:305-306`）。没有做切图或 OCR 预处理，原因有两个：日程截图通常信息密度不高，整图能一次看完，切图反而会切断上下文；预处理链路越长，出错的地方越多。

提示词针对截图场景做了专门约束：课表按「科目 + 时间 + 地点」逐条列；无法确定年份时按今年处理；只写截止日的跨日事项也要置 `isDeadline=true`（`ai-core.ts:64-72`）。这三条都是实际导入课表截图时暴露的问题。

9.5 三种导出格式

格式	入口	用途
应用 JSON	`/api/export?format=json`	备份与迁移，可被本地模式原样导回
iCal	`?format=ical`	导入系统日历 / 手机日历，生成 `VTODO` 与 `VEVENT`
课程表结构	`?format=sched`	与另一套「日程表」作品互通，带学期起始日

表 9-1 三种导出（`src/app/api/export/route.ts:21-130`）

三种导出都排除垃圾箱里的数据（`export/route.ts:91`）。这个细节容易被忽略：如果不排除，用户导出的备份里会混进几十条已删除的旧任务，再导回来时全部复活。

9.6 分享长图：借系统浏览器渲染

「导出图片」不是 canvas 画出来的，而是拼一段 HTML、用无头浏览器截图。这样做的好处是 可以直接复用 CSS（渐变、圆角、阴影），不必用 canvas API 重新实现一遍视觉。代价是依赖一个 Chromium 内核的浏览

器。

查找顺序是：环境变量 `SMART_CHROME_PATH` → `MSEDGE_PATH` → 各平台默认路径（`export/image/route.ts:16-49`）。Windows 安装包内置了 `chrome-headless-shell`，启动器会把它的路径注入环境变量，所以用户机器上即使没装浏览器也能导出。

```
const args = ["--headless=new", "--no-sandbox", "--disable-gpu", "--hide-scrollbars",
  "--disable-extensions", "--virtual-time-budget=1200",
  `--screenshot=${out}`, `--window-size=900,${height}`, url];
```

代码 9-2 截图参数（`export/image/route.ts:203-210`，数组已折行）

`--no-sandbox` 是必需的。内置的 `chrome-headless-shell` 以受限权限运行时，缺少这个参数会直接崩溃，且不输出任何有用日志——这个坑在集成阶段花了半天才定位。高度按条目数估算，公式是 `min(4200, max(480, 340 + 条数 × 86))`，上限 4200 是为了避免生成一张几十兆的图。

9.7 导入与导出并不完全对称

三种导出格式里，只有应用 JSON 能原样导回。这不是偷懒，而是三种格式的能力边界不同：

- **应用 JSON**：结构就是 `{ todos: [...] }`，本地模式可直接解析，字段一一对应，包含课程标签与子任务。
- **课程表结构**：额外带 `semester.week1Start`，导回时会被识别，并写回本地教学周历设置（`ImportModal.tsx:131-137`）。
- **iCal**：单向。ICS 的重复规则（`RRULE`）与本地模型不兼容——本地一条日程只表达一个时间点或一段区间，没有「每周三重复」的概念，硬导进来会展开成无限多条。

跨设备迁移用 JSON，跨应用用 iCal，这个分工写在导出菜单的说明里，避免用户拿 iCal 备份再来问为什么丢了标签。

10 三平台打包与本地化

BUILD · SEA · INSTALL

> 开发时 `npm run dev` 就能跑，但交付给一个没有 Node、没有命令行的用户， 中间隔着构建、注入、安装、数据目录、端口五道关。这一章逐关说明。

10.1 第一阶段：构建出可搬运的目录

`next.config.mjs` 里有两个关键开关（`next.config.mjs:5-11`）： `output: "standalone"` 让构建把运行期依赖收进一个目录； `distDir` 可被 `NEXT_DIST_DIR` 覆盖，便于每次构建换一个输出目录。

```
const nextConfig = {
  output: "standalone",
  distDir: process.env.NEXT_DIST_DIR || ".next",
  compiler: { removeConsole: process.env.NODE_ENV === "production" },
};
```

代码 10-1 构建配置（`next.config.mjs:4-12`，省略图片与实验项）

换输出目录不是洁癖。Windows 上 `.next/trace` 经常被杀毒软件或残留进程占用， 第二次构建直接报 `EPERM`；同时 `tsconfig.json` 的 `include` 会累积每次构建的类型路径，删掉旧目录后不清理就会引用不存在的类型。 现在每次构建用一个新目录（`.next33` 这样的编号），问题消失。

构建完成后还有一步手工补齐：`static` 与 `public` 不会自动进 standalone 目录 （`README.md:89-93`）。少了 `static` 页面没有样式，少了 `public` 图标与 manifest 全部 404。

10.2 第二阶段：SEA 把启动器塞进 node.exe

用户机器上没有 Node，所以要把一个 Node 运行时和启动脚本合成一个 exe。Node 官方的 SEA 分三步：写配置、生成 blob、注入。配置只有五行（`build-desktop/sea-config.json`）：

```
{ "main": "launcher.js", "output": "sea-prep.blob",
  "disableExperimentalSEAWarning": true }
```

代码 10-2 SEA 配置（`build-desktop/sea-config.json`）

图标要在注入前写进 exe 的资源段，用的是 `resedit`： 复制一份 `node.exe`，替换 `IconGroup 101/1033`， 再写入版本信息（`build-desktop/apply-icon.cjs:10-37`），最后用 `postject` 把 blob 注入 `NODE_SEA_BLOB` 段。

校验

注入是否成功不能看命令返回值。曾经出现过命令返回 0、产物只有 85 MB 的情况—— `postject` 没报错但 blob 没进去，运行时报「找不到入口」。 现在的做法是：注入后先比对体积（应大于 160 MB），再在临时目录跑一次完整安装与启动， 两条都通过才允许覆盖成品。

10.3 第三阶段：自解压安装器

安装器本身也是一个 SEA 程序，只是它的资产里多了一个 `payload.zip`（约 200 MB，含 app 目录、node.exe、内置浏览器）。运行时按六步执行，每一步都打印进度，方便用户在双击后看到「没卡住」：

步骤	动作	关键处理
1/6	释放 payload 到临时目录	`sea.getAsset("payload")`，打印 MB 数
2/6	解压	调用 PowerShell `Expand-Archive`，失败可重试
3/6	停止旧版本	按命令行匹配 `SmartSchedule*server.js` 杀进程
4/6	复制文件	整体替换 `app/`，可选复制 `resources/`
5/6	快捷方式与卸载项	桌面 + 开始菜单，注册表写 `Uninstall` 项
6/6	放行防火墙	`netsh` 先删同名规则再加 3100 端口

表 10-1 安装器六步（`installer/sea-installer.js:44-174`）

第 3 步是「升级不失败」的关键。旧版本还在跑的时候，`app/` 里的 `server.js` 与 `node.exe` 被占用，覆盖会直接抛 `EBUSY`。先杀进程再复制，失败也不阻塞（`allowFail`），最坏情况是留下旧文件，由下一次安装覆盖。

安装器还留了四个开关供自动化验收：`SMART_INSTALL_DIR`（改安装目录）、`SMART_NO_SHORTCUT`、`SMART_NO_LAUNCH`、`SMART_NO_FIREWALL`。验收脚本用它们把包装到临时目录、不建快捷方式、不自动启动，跑完直接删——不污染开发机。

10.4 数据目录必须住在安装目录之外

这是打包环节最容易犯的错。Next standalone 启动时会把工作目录切到 `app/`，而仓储的路径是 `path.join(process.cwd(), "data")`（`repository.ts:10`），于是数据落在 `app/data`。升级时整个 `app/` 被替换，用户几个月的日程一起消失。

解决办法是让 `app/data` 变成一个指向外层 `data` 的联接：安装器在复制完成后删除 `app/data` 并创建 junction（`sea-installer.js:82-85`），启动器每次启动再自检一遍（`launcher.js:22-60`）。自检逻辑分三种情况：

- 已经是符号链接：什么都不做。
- 是普通目录（老版本遗留）：逐个文件搬到外层，同名时比 `mtime` 决定是否覆盖，然后删目录。
- 建联接失败：退回 `app/data`，至少能跑，日志里留一行。

macOS 与 Linux 用符号链接实现同一件事，路径按平台惯例放：`~/Library/Application Support/SmartSchedule` 与 `${XDG_DATA_HOME:-~/.local/share}/SmartSchedule`（`mac-launcher.sh:10-13`、`linux-start.sh:11-14`）。

10.5 端口：先确认「是不是我」

默认端口 3100。用户机器上 3100 很可能被别的开发服务占着，如果直接启动会失败，如果直接顺延又可能重复拉起第二个实例。所以判断逻辑必须是两段式（`launcher.js:102-109`）：

```
async function pickPort() {
  for (let p = BASE_PORT; p < BASE_PORT + 10; p++) {
    if (await ping(p)) return { port: p, running: true };   // 是我们的服务 → 复用
    if (!(await portOpen(p))) return { port: p, running: false }; // 空闲 → 启动
  }
  return { port: BASE_PORT, running: false };
}
```

代码 10-3 端口选择（`build-desktop/launcher.js:102-109`）

`ping` 不只看 HTTP 200，还要求响应体里 `app === "smart-schedule"`（`launcher.js:66-85`）。少了这个指纹，任何返回 200 的服务都会被当成自己，结果是「打开了浏览器，但显示的是别人的页面」——这正是白屏反馈的根因之一。

确认端口后，服务以 `HOSTNAME=0.0.0.0` 启动，同一局域网的手机才能访问。浏览器打开的是 `localhost`，二维码里编的是局域网 IP，两者共用同一个进程。

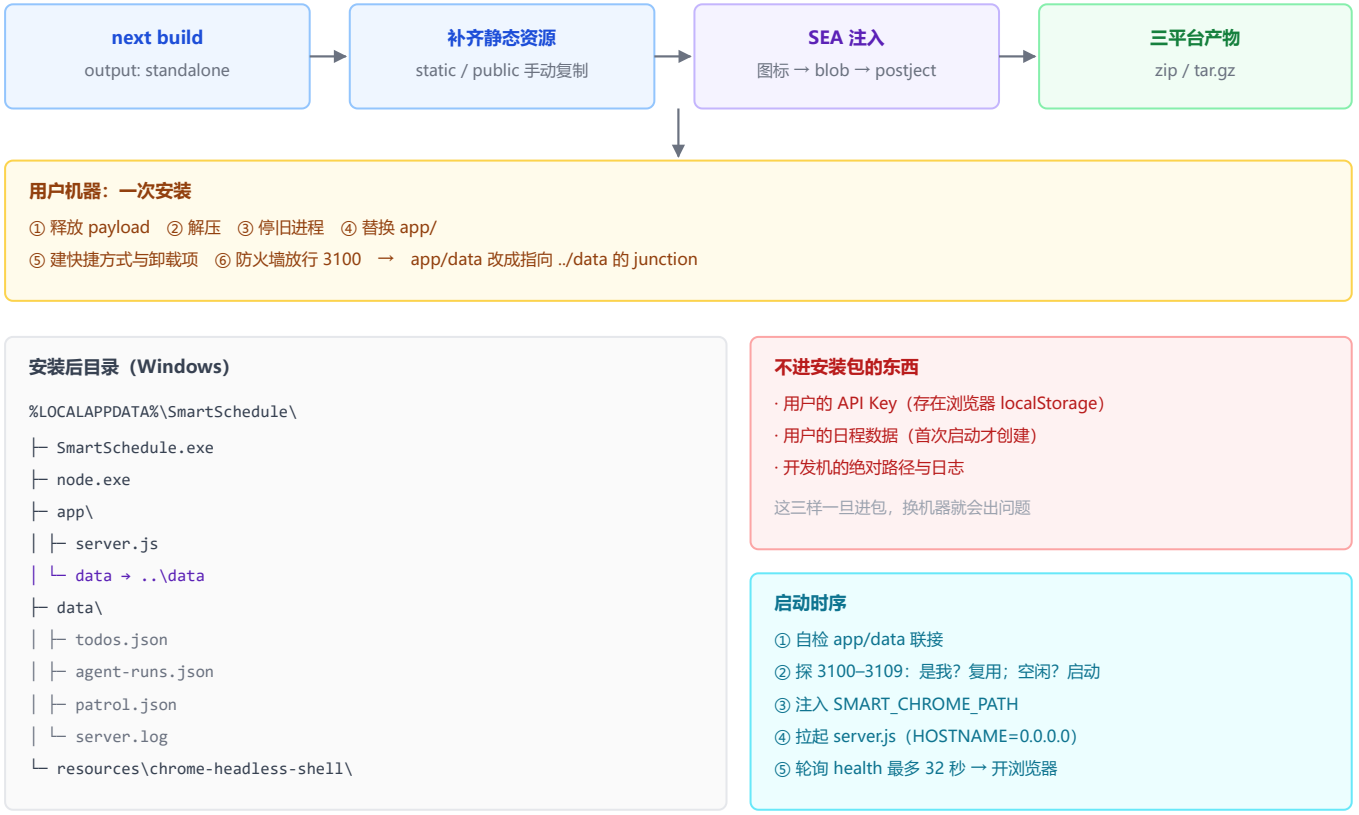


图 10-1 从源码到安装的完整链路。红框标出「绝不进包」的三类内容，右下是安装后的启动时序。

10.6 三个平台的差异

维度	Windows	macOS	Linux
入口	`SmartSchedule.exe`（SEA）	`.app/Contents/MacOS/` 下的 bash	`start.sh`
运行时	单个 `node.exe`	双 Node（arm64 + x64），按 `uname -m` 选	单个 ELF
数据目录	`%LOCALAPPDATA%`	`~/Library/Application Support`	`~/.local/share`（遵循 XDG）
图标	resedit 写入 exe 资源	转 `.icns`，写 `Info.plist`	PNG 256×256，`.desktop` 绝对路径
权限	无需处理	zip 逐文件写 `external_attr` 保 755	tar 统一 uid/gid 与 755/644
卸载	注册表 Uninstall 项 + 脚本	拖到废纸篓	删除安装目录 + `.desktop`
离线浏览器	内置 `chrome-headless-shell`	依赖系统 Chrome/Edge	依赖系统 Chrome/Edge

表 10-2 平台差异（`build-desktop/build-platforms.py`、各平台启动脚本）

双 Node 是 macOS 特有的麻烦：Intel 机器与 Apple Silicon 不能共用同一个二进制。 包内放两份，启动脚本按 `uname -m` 选择，代价是体积翻倍（约 86 MB）， 换来的是「一个包通吃」——用户不需要知道自己是什么芯片。

zip 的 `external_attr` 值得单独提一句：Python 的 `zipfile` 默认不保留 可执行位，解压后启动脚本没有 `x` 权限，双击直接失败。写 zip 时必须 `external_attr = (mode & 0xFFFF) << 16`，tar 则用 filter 统一权限位。这两个细节不写，产物在别人机器上就是「打不开」，而本地测试完全正常。

10.7 出问题时看哪里

安装后的所有输出都写进 `data/server.log`（`launcher.js:122-123`）， 包括端口选择、服务退出码、启动失败原因。用户反馈「打不开」时，先看这个文件的前 20 行， 多数情况能直接判断是端口被占、数据目录被占用，还是 Node 被杀毒软件拦截。

11 工程决策与踩坑记录

TRADE-OFFS · PITFALLS

> 技术文档最容易漏掉的是「为什么没选另一条路」，以及「当时到底错在哪」。 这一章把八个关键决策的代价、十个踩过的坑、一份可复现的验收清单放在一起。

11.1 八个关键决策

决策	选择	代价	结论
数据存哪	本地 JSON 文件 + 可选云库	无并发控制、无事务	单用户场景够用，换来实现「拷走就能带走」
Key 存哪	浏览器 localStorage，随请求下发	换浏览器要重填；前端可读	保证安装包不含密钥
桌面形态	Node + 系统浏览器，不用 Electron	无托盘、无原生菜单	体积从 80 MB 降到可用水平
Agent 边界	6 步 / 90 秒 / 30k tokens 硬上限	复杂任务会被截断	可预期比强大更重要
删除权限	模型只能提议，人确认后软删	多一次点击	不可逆操作不接受自动化
富文本	自研 231 行渲染器	不支持表格/代码块	层级可控，体积可控
安装器	Node SEA 自解压，弃用 IExpress	要自己写六步流程	IExpress 静默安装会卡死且无日志
图标	一个 ICO 派生 exe / icns / png	每次构建多一步转换	三平台视觉一致

表 11-1 关键决策与代价

其中最值得复盘的是安装器。最初用 Windows 自带的 IExpress 做自解压包， 配置简单、双击即用，但静默安装会卡在某个对话框上，而且没有任何日志可查。 换成 Node SEA 之后，安装过程变成一个可以 `console.log` 的普通程序， 每一步都能验收——「可观测」本身就是选择它的理由。

11.2 十个易错点

#	现象	根因	处理
1	构建报 `EPERM: .next/trace`	上次构建的进程或杀软仍占用文件	每次用新的 `NEXT_DIST_DIR`，并清理 tsconfig 的旧类型路径
2	提示词一改就构建失败	模板字符串里出现裸反引号，提前结束字符串	提示词正文不用反引号，示例 JSON 直接写
3	导入接口编译报 `mode` 未定义	`let mode` 声明在 `try` 内，作用域出不去	提到函数顶层（`parse/route.ts:313`）
4	批量创建 0 条、失败 2 条	可选时间字段没传被判为「格式错误」	只在显式传值时才校验（`tools.ts:585-592`）
5	截图导出直接崩，无日志	内置 `chrome-headless-shell` 缺 `--no-sandbox`	公共参数硬编码该开关（`export/image/route.ts:204`）
6	压缩包时有时无	压缩与重命名并行，目标文件还没写完	全部改串行：先清目录，再逐平台写盘
7	升级后日程全没了	数据写在 `app/data`，被整体替换	安装器重建 junction，启动器自检迁移
8	打开是别人的页面 / 白屏	只探端口不探身份，3100 被别的服务占用	校验 `/api/health` 的 `app` 指纹
9	安装包只有 85 MB，运行报错	`postject` 未真正注入 blob，但返回 0	注入后校验体积 > 160 MB，并跑一次端到端安装
10	macOS 上双击无反应	zip 丢了可执行位	写 zip 时保留 `external_attr`，tar 统一 755

表 11-2 十个易错点及其证据位置

这十条有一个共同点：**本地测试全部正常，换一台机器或换一个时序才暴露。** 所以第 7、9 条最终都补了「在临时目录做一次完整安装」的验收步骤，而不是相信构建日志。

11.3 可复现验收清单

发布前按顺序执行，每条都有明确的通过标准：

1. **构建**：`npm run build` 成功，产物目录里有 `server.js` 与 `static/`。
2. **裸跑**：直接 `node app/server.js`，`/api/health` 返回 `app: "smart-schedule"`。
3. **安装**：`SMART_INSTALL_DIR=临时目录 SMART_NO_SHORTCUT=1 SMART_NO_LAUNCH=1` 静默安装，退出码为 0。
4. **数据外置**：安装目录下 `app/data` 是链接，`data/todos.json` 存在。
5. **升级不丢数据**：写入一条测试数据，重装一次，数据仍在。
6. **Agent 链路**：`AGENT_MOCK=1` 跑一次对话，返回 `stopped_reason: "final"` 且轨迹里有工具调用。
7. **撤销**：用 `batch` 假模型创建两条，再撤销，条数回到初始值。
8. **导出**：`/api/export/image?range=week&format=png` 返回 > 100 KB 的 PNG。

经验

验收要在「没有开发环境的目录」里做。开发机上 `node` 在 PATH 里、`.env.local` 存在、端口空闲，很多问题会被掩盖。临时目录 + 干净环境变量，才能复现用户看到的样子。

12 二次开发指南

EXTEND · REPLACE

> 如果要把这个项目接着做下去，最常改的三件事是：给 Agent 加一个工具、换一个存储后端、支持一种新文件。 每一步的改动点都固定，下面按顺序列清楚。

12.1 新增一个 Agent 工具

以「给多条日程批量加标签」为例，需要改五处，缺一处就会出问题：

1. **类型**：在 `ToolName` 联合类型里加 `"tag_many"`（`tools.ts:10-19`）。
2. **Schema**：在 `TOOL_SCHEMAS` 里加一项，写清参数与必填项（`tools.ts:72-251`）。模型靠这段描述决定是否调用，描述要比代码注释更啰嗦。
3. **执行器**：在 `executeTool` 的 `switch` 里加 `case`，参数校验失败时返回 `{ ok:false, error, summary }`，不要 `throw`。
4. **还原点**：每个写操作都要 `ctx.mutations.push({ type:"update", id, before: snapshotOf(prev) })`（`tools.ts:401-403`），否则这次改动作无法撤销。
5. **只读判定**：如果是只读工具，把它加进 `READONLY_TOOLS`（`runtime.ts:102`）；如果是写工具，什么都不用做，只读模式下会被自动拦截。

```
case "tag_many": {
  const ids = Array.isArray(args.ids) ? args.ids.map(String).slice(0, 30) : [];
  const tags = Array.isArray(args.tags) ? args.tags.map(String).slice(0, 6) : [];
  if (!ids.length || !tags.length) return { ok: false, error: "ids 与 tags 必填", summary: "加标签失败"
};
  let n = 0;
  for (const id of ids) {
    const prev = await repository.get(id);
    if (!prev) continue;
    ctx.mutations?.push({ type: "update", id, before: snapshotOf(prev) });
    await repository.update(id, { tags: [...new Set([...prev.tags, ...tags])] });
    n++;
  }
  return { ok: true, data: { updated: n }, summary: `已为 ${n} 条日程加标签` };
}
```

代码 12-1 一个写工具的完整骨架（对齐 `tools.ts` 现有风格）

写完之后做两件事：在提示词的「可用工具」清单里补一行（只在 JSON 降级模式下需要， 原生 tools 模式 schema 会自动下发），并在评测集里加一条用例，断言 `steps` 里出现该工具名且 `stopped_reason === "final"`。

12.2 替换存储后端

新后端不需要实现任何接口，只要提供同名方法，然后在 `server/index.ts` 改一行分发。 需要实现的 11 个方法是：

方法	签名要点	语义
`list()`	→ `Todo[]`	按 `order` 升序，含垃圾箱
`get(id)`	→ `Todo | null`	不存在返回 null，不抛错
`create(input)`	`TodoInput` → `Todo`	order 取当前最小值减一
`bulkCreate(inputs)`	→ `Todo[]`	一次写入完成 N 条
`update(id, patch)`	→ `Todo | null`	含状态与完成时间联动
`restore(id, snapshot)`	→ `Todo | null`	不做联动，用于撤销
`remove(id)`	→ `boolean`	物理删除，返回是否存在
`removeMany(ids)`	→ `number`	返回实际删除条数
`clear()`	→ `number`	清空全部，返回删除条数
`emptyTrash()`	→ `number`	只删 `trashedAt` 非空的
`autoMaintain()`	→ `{ todos, trashed, progressed, completed }`	三段式规则，顺序不可换

表 12-1 存储后端契约（对照 `repository.ts:80-242`）

最容易漏的是 `restore` 与 `autoMaintain`。前者容易被当成 `update` 的别名实现，导致撤销后时间戳被改写；后者容易被简化成「只做自动完成」，丢掉 30 天入箱规则。 两个方法都必须按语义实现，不能近似。

12.3 新增一种导入格式

改动点只有三处，集中在 `import/parse/route.ts`：

1. 在 `POST` 的扩展名分支里加一个 `if`，`e` 是去掉点的小写扩展名。
2. 写一个 `parseXxx(buf, ai)`，返回 `{ todos, note }`；内部要么用规则解析，要么调用 `extractTodosWithAI`。
3. 更新 `ImportModal` 的 `accept` 列表与错误文案（AI 模式与本地模式各一份）。

注意本地模式不需要改：它只接受 JSON，其他扩展名统一返回 415。 这是有意的收窄，不是遗漏。

12.4 改提示词的三条纪律

第一，不要用反引号。提示词是模板字符串，示例里的 `` ` `` 会截断字符串。 需要强调代码时用中文引号或直接写。

第二，结构化输出要求要和渲染器对齐。提示词里要求的 `##`、`-`、`1.`、`> 建议：` 都在 `RichText` 的解析范围内。 如果要求模型输出 `### 三级标题` 却忘了渲染器只映射到 h2，层级就会塌。

第三，任何新增约束都要有对应断言。提示词是软约束，验证只能靠评测用例： 改完跑 `node scripts/eval-agent.mjs`，看 14 条里有没有退化。 没有断言的约束，下一次改提示词时就会悄悄失效。

12.5 调试与自测的顺序

改动之后不要直接找真实模型验证。按下面五步依次排查，多数问题在前两步就能定位：

检查项	看什么	常见原因
`/api/health`	返回的 `app` 是否为 `smart-schedule`	端口被别的服务占用，或进程根本没起来
`data/server.log`	最后 20 行：端口选择、退出码	端口顺延失败、Node 被杀毒软件拦截
模拟模式	`AGENT_MOCK=1` 起服务，跑 `eval-agent.mjs --mock`	工具、熔断、撤销的能力回归
`/api/ai/test`	延迟与返回的模型名	Key 无效（401）、模型名写错（404）
导出长图	返回的 PNG 是否大于 100 KB	找不到浏览器（501），或缺少 `--no-sandbox`

表 12-2 排查顺序与判断依据

这个顺序的意义是「把变量分开」：先确认进程，再确认链路，最后确认模型。反过来做，一次失败会让人同时怀疑三件事，排查时间会成倍增加。

```
# 1) 探活：确认端口上是本应用
curl -s http://localhost:3100/api/health

# 2) 无 Key 自测：模拟模式起服务，再跑评测集
AGENT_MOCK=1 node app/server.js &
node scripts/eval-agent.mjs --mock

# 3) 换真实模型：先测连通性，再跑一次对话
curl -s -X POST http://localhost:3100/api/ai/test -H "Content-Type: application/json" \
  -d '{"baseUrl":"https://api.deepseek.com/v1","apiKey":"sk-...","model":"deepseek-chat"}'
```

代码 12-2 三步自检命令（Windows 下把续行符 `\` 去掉、写成一行）

13 增补：后台提醒与覆盖安装

REMINDERS · INSTALLER CLEANUP

> 这一章是 v1.1 的增补。前面十二章描述的是「记录与执行」，本章补上两件交付层面的事： 到点主动提醒，以及新版本安装时把旧版本清干净。两者都不改核心数据模型，但都要和既有边界对齐。

13.1 提醒规则：三条时间线

提醒不是「到点响一下」这么简单。日程有两种形态，时间点也不同，规则必须写死才不会出现 「同一件事弹两次」「没写时间也弹」这类问题。最终确定的规则只有三条：

类型	触发点	提前量
普通日程	开始时间（`dueDate` + `dueTime`）	60 分钟
截止事项 · 开始	`startDate`（无则 `dueDate`）+ `dueTime`	60 分钟
截止事项 · 结束	`dueDate` + `endTime`	180 分钟

表 13-1 提醒规则（`src/lib/server/reminders/engine.ts`）



图 13-1 两类日程的提醒时间线。灰色虚线表示提前量，下方列出所有「不提醒」的例外。

规则写成纯函数 `planReminders(todo, settings)`，不碰文件、不碰网络，因此可以单独测。 它只回答「这条日程应该在哪两个时刻提醒」，是否已经触发、是否错过，由上层判断。 这样做的好处是：新增一条规则（比如「全天事项早上 9 点提醒」）只需要改这一个函数。

```
const startAt = parseLocalDateTime(todo.startDate || todo.dueDate, todo.dueTime);
if (startAt !== null) out.push({ kind: "start", at: startAt, fireAt: startAt - leadStart * MIN, leadMin: leadStart });
const endAt = parseLocalDateTime(todo.dueDate, todo.endTime);
if (endAt !== null) out.push({ kind: "end", at: endAt, fireAt: endAt - leadEnd * MIN, leadMin: leadEnd });
```

代码 13-1 截止事项的两个提醒点（`engine.ts`，省略去重）

时间解析用本地时区，并额外校验「2026-02-31」这类会被 `Date` 自动进位的非法日期， 不合格就返回 `null`，该条提醒不生成——宁可少提醒，也不要在错误的时间弹。

13.2 送达：服务端排队，浏览器负责显示

本地应用没有公网推送通道，也没有推送服务，所以采用「服务端算、客户端弹」的分工： 服务端每 20 秒扫描一次，把到点事件写进 `data/reminders.json` 的 pending 队列； 页面每 15 秒（以及重新可见、窗口获得焦点时）拉取一次，交给 Service Worker 弹系统通知， 成功后回执，服务端记录送达通道。

环节	周期	职责
`scheduler.ts`	20 秒	扫描全部日程 → 入队 → 清理过期排队项
`ReminderCenter`	15 秒 + 可见性变化	拉取 pending → 弹窗 → 回执
`sw.js`	事件驱动	`showNotification` 与点击聚焦窗口
`/api/reminders`	请求驱动	读取状态、确认、测试、修改开关

表 13-2 四个环节的职责（`scheduler.ts` / `ReminderCenter.tsx` / `sw.js` / `api/reminders`）

```
const reg = await ensureServiceWorker();
if (reg) {
  await reg.showNotification(e.title, { body: e.body, tag: e.id, icon: "/icons/icon-192.png", data: {
url: "/" } });
  return "sw";
}
```

代码 13-2 优先用 Service Worker 弹通知（`src/lib/reminder-client.ts`）

用 Service Worker 而不是页面 `Notification`，是因为页面切到后台后仍由浏览器进程 统一把通知交给系统通知中心，点击还能聚焦回窗口。权限被拒绝时降级为页面通知， 再不济就只显示应用内浮层——三层降级保证「至少能看到」。

两个时间参数是刻意设置的：服务端 20 秒是为了让「1 小时前」的误差不超过 20 秒； 客户端 15 秒是为了让后台标签在浏览器节流下仍能及时取到。排队项超过 30 分钟未送达即作废， 避免用户第二天打开应用时被十几条旧提醒轰炸。

13.3 覆盖安装：先清干净，再装新的

旧版安装器只覆盖同名文件，旧版本里被删掉的文件会一直留在磁盘上（例如旧组件、旧资源）。 v1.1 在复制之前加了一步「清理旧版本」，规则是：**除 `data/` 以外，安装目录里的东西全部删除。**

步骤	动作	关键处理
1	识别目录	空目录=全新安装；含 exe/app/标记=本应用；否则中止，不动任何文件
2	拆联接	`app/data` 若是指向 `../data` 的 junction，先解除，防止递归删除穿透
3	删除旧文件	除 `data/` 外全删；删不掉的改名 `.old-<ts>-*` 隔离
4	复制新文件	复制 app、node、启动器、resources
5	重建联接与标记	重建 `app/data`，写安装标记与 `data/install.log`

表 13-3 清理五步（`installer/sea-installer.js` 第 4 步）

```
for (const name of fs.readdirSync(dir)) {
  if (KEEP_IN_TARGET.has(name) || name.startsWith(".old-")) continue;
  const p = path.join(dir, name);
  if (name === "app") detachAppData(dir);
  if (rmWithRetry(p)) continue;
  fs.renameSync(p, path.join(dir, ".old-" + Date.now() + "-" + name));
}
```

代码 13-3 清理主循环（`clearOldVersion`，省略计数与异常分支）

三个护栏是这段代码能上线的前提：拒绝盘符根目录与一级目录；拒绝不含安装标记的目录； 先拆联接再删目录。少了任何一条，一次误操作就可能删掉用户的文档目录或日程数据。 被占用而删不掉的文件不阻塞安装，改名隔离并在下次安装开头再清，日志里能看到具体是哪一个。

Linux 的 `linux-install.sh` 采用同样语义（`app.old` 改名 → 复制 → 删除）， macOS 由 Finder 整体替换 `.app`，用户数据在 `~/Library/Application Support`，天然不在替换范围内。

13.4 两轮自检

两个功能各做两轮验证，全部通过。自检脚本留在仓库里，改完代码可以直接复跑。

功能	轮次	断言数	结果	覆盖内容
覆盖安装	第一轮	11	全通过	同目录升级、旧文件清除、数据保留、联接重建、装后启动
	第二轮	9	全通过	全新安装、幂等重装、误目录保护、遗留隔离目录清理
后台提醒	第一轮	16	全通过	60/180 分钟边界、无时间、完成/免打扰/垃圾箱、宽限、去重、排队过期
	第二轮	11	全通过	真实浏览器后台标签、Service Worker 通知、送达通道、设置页

表 13-4 自检结果（`installer/selftest-install.mjs`、`scripts/reminders-*.mjs`）

第一轮的提醒用例里有两条专门盯边界：提前 61 分钟不触发、60 分钟触发； 截止事项在结束前 180 分钟再触发一次。第二轮的浏览器用例把页面切到后台标签， 确认 `visibilityState` 为 `hidden` 时仍能收到两条通知， 并用 `getNotifications()` 证实通知确实由 Service Worker 创建。

14 多主题系统：一份组件，四套皮肤

THEMING · ACCESSIBILITY

> v1.2 加了主题切换：标准、青少年、老年、二次元四套。同一个页面、同一套交互， 只换颜色、字号、间距与动效。难点不在“配色好看”，而在于不把两百多个组件逐个改一遍， 还要让老年版真的可用、让二次元不变成壁纸。

14.1 四套主题各自解决什么

主题	面向	关键差异
标准版	日常与夜间	深色极光背景，信息密度最高，默认主题
青少年版	学生、年轻用户	奶油底 + 珊瑚红，贴纸式卡片（2px 描边 + 硬阴影），胶囊按钮
老年版	视力与操作精度下降	根字号 19px，正文约 21px；2px 深色描边；点击区 ≥50px；无动画无模糊；看板与列表单列；三步指引常驻
二次元版	偏好动漫风格	浅蓝天空 + 奶油卡片 + 金色描边，背景四角星星光，圆角 22px

表 14-1 主题与设计取舍（`src/lib/theme.ts`）

14.2 实现：变量打底，重映射收尾

项目早期是纯深色界面，组件里散落着 `text-white`、`bg-white/10`、`bg-ink-800` 这类硬编码颜色。全量重构风险太大，所以采用两层策略： 第一层把语义色收敛成 CSS 变量（背景、面板、文字、边框、强调色、圆角）， 第二层用**作用域属性选择器**把这些历史工具类在浅色主题下重新映射。

```
html[data-theme="teen"] {
  --bg: #fff4e6;  --panel: #ffffff;  --text: #1b1033;
  --border: #241546;  --accent: #d32f3c;  --radius: 22px;
}
/* 一行覆盖全站硬编码的白色文字，只在新主题下生效 */
html[data-theme="teen"] [class*="text-white"] { color: var(--text) !important; }
html[data-theme="teen"] [class*="bg-white/"]:not([class*="hover:bg-white/"]) {
  background-color: rgba(27, 16, 51, 0.06) !important;
}
```

代码 14-1 变量 + 作用域重映射（以青少年版为例，省略其余变量）

`:not([class*="hover:bg-white/"])` 这个细节很关键：如果不排除 `hover:` 前缀， 所有带 hover 背景的导航项都会常驻底色，当前页反而看不出来。 加上之后，选中态才由真正的 `bg-white/10` 承担。

这样新增一套主题的成本变成“写一段 CSS 变量 + 少量组件覆盖”， 而不是“改一遍所有组件”。四套主题共用同一套 DOM 与交互，功能天然一致。

14.3 切换入口与持久化

入口	位置	说明
侧栏主题按钮	桌面左侧栏顶部	显示「主题 · 当前名称」，点开三/四个选项
顶栏调色板	移动端顶栏	紧凑模式，只显示图标
设置 → 外观	设置抽屉第 4 个页签	色板预览 + 适用人群 + 生效范围说明

表 14-2 三个入口共用同一状态（`ThemeSwitcher.tsx`、`ThemeSettings.tsx`）

选择写入 `localStorage["scheduleApp.theme.v1"]`。 为了避免首屏闪白/闪黑，在 `<head>` 里放一段同步脚本， 在 React hydrate 之前就给 `<html>` 打上 `data-theme`：

```
<script>{`(function(){try{
  var t=localStorage.getItem('scheduleApp.theme.v1')||'standard';
  if(t!=='teen'&&t!=='senior'&&t!=='anime')t='standard';
  document.documentElement.dataset.theme=t;
}catch(e){document.documentElement.dataset.theme='standard';}})()`}</script>
```

代码 14-2 首屏主题脚本（`app/layout.tsx`，省略 colorScheme 设置）

14.4 二次元星光：不引入一张图片

二次元主题的星光用四层内联 SVG 数据 URI 铺成：每层是一张 240×240 的画布， 中央放一个四角星，再按 260–520px 的间距平铺。四层的颜色、大小、透明度不同， 叠起来就有疏密变化，而不是重复的壁纸纹理。

```
url("data:image/svg+xml,%3Csvg ... viewBox='0 0 240 240'%3E
  %3Cg transform='translate(120,120) scale(0.42) translate(-32,-32)'%3E
    %3Cpath d='M32,2C34,26 38,30 62,32 38,34 34,38 32,62 30,38 26,34 2,32 26,30 30,26 32,2Z'
          fill='%23ffffff' fill-opacity='.95'/%3E
  %3C/g%3E%3C/svg%3E")
```

代码 14-3 四角星图形（曲线星形，非五角星）

第一版把 `background-size` 设成 18–34px，本意是星星大小， 结果平铺密度等同于壁纸，文字背后全是花纹。改成 240px 画布 + 260–520px 平铺后， 每屏只有十几颗星，才既有装饰感又不干扰阅读。整体透明度在 0.5–0.8 之间做 9 秒呼吸， 并遵守 `prefers-reduced-motion`。

14.5 无障碍与响应式

- **对比度**：老年版正文 `#111` / `#FFFDF6` 约 18:1，次要文字 `#333` 约 12:1；青少年与二次元正文约 11:1。
- **优先级徽标**：浅色主题下改为深色字 + 浅底 + 同色描边，解决原来亮绿字在白底上看不清的问题。
- **焦点态**：全局 `:focus-visible` 3px 描边，老年版 4px；键盘可走完整个流程。
- **触控**：老年版按钮最小高度 54px，图标按钮 ≥46px；日历上/下月按钮最小宽度 58px。
- **布局**：老年版看板与列表塌缩为单列、统计两列；日历 7 列外包横向滚动，窄屏不压扁格子。
- **动效**：老年版关闭动画与毛玻璃；其余主题尊重系统的减少动态偏好。

14.6 验证

范围	项数	覆盖
二次元专项	8	主题生效、看板三列、新建日程、列表/统计、刷新保持、数据在库、外观页四个选项
三套主题回归	24	每套主题重复上述 8 项；另验老年版单列与 19px 字号
截图留证	8 张	四套主题的看板与外观页（`deliverables/theme-proof/`）

表 14-3　主题自检结果（`scripts/theme-smoke.mjs`）

A 附录：接口、环境变量与文件清单

APPENDIX

> 附录用于检索，不做解释。所有条目按代码实际导出整理，与正文的 `file:line` 引用互为索引。

A.1 接口全表

方法	路径	入参	返回
GET	`/api/todos`	`status` `priority` `tag` `q` `all` `trash`	`{todos,total}`
POST	`/api/todos`	`action` / `items[]` / `title`	`{todo}` 201、`{todos,count}`、`{removed}`
GET	`/api/todos/[id]`	路径 id	`{todo}` / 404
PATCH	`/api/todos/[id]`	任意可改字段 + `order`	`{todo}` / 404
DELETE	`/api/todos/[id]`	路径 id	`{ok:true}` / 404
GET	`/api/agent`	—	`{limits,tools,mock_enabled}`
POST	`/api/agent`	`instruction` ≤1000 字、`history` ≤6、`ai`	`AgentRun`（含 `stopped_reason`）
GET	`/api/agent/patrol`	—	`{cached,stale}`
POST	`/api/agent/patrol`	`ai`、`force`	`{date,summary,engine,generated_at}`
POST	`/api/agent/undo`	`run_id`	`{ok,undone,failed[],details[]}`
POST	`/api/ai/chat`	`mode`（5 种）、`text` / `todos` / `messages`、`ai`	`{engine,model,keySource,mode,...}`
POST	`/api/ai/test`	`baseUrl`、`apiKey`、`model`	`{ok,latency,model,reply}`
POST	`/api/import/parse`	multipart: `file`、`mode`、`ai`	`{todos,note?,semester?}`
GET	`/api/export`	`format=json|ical|sched`、`week1Start`	附件流
GET	`/api/export/image`	`range=week|today|all`、`format=png|pdf`	`image/png` / `application/pdf` / 501
POST	`/api/auto`	—	`{todos,trashed,progressed,completed}`
GET	`/api/network`	—	`{port,urls[],interfaces[],hostname}`
GET	`/api/qr`	`text`（≤300 字）	`image/svg+xml`
GET	`/api/reminders`	—	`{settings,pending,log,counts,scheduler}`
POST	`/api/reminders`	`action=ack|test|settings|tick`	`{ok,acked|event|settings|result}`
GET	`/api/health`	—	`{app,ok,version,platform,arch}`

表 A-1 21 个处理函数（15 个 `route.ts`）

A.2 环境变量

变量	默认	作用
`OPENAI_API_KEY`	空	服务端兜底 Key；用户没在界面填时使用
`OPENAI_BASE_URL`	`https://api.openai.com/v1`	OpenAI 兼容端点
`OPENAI_MODEL`	`gpt-4o-mini`	模型名
`CLOUD_ENDPOINT`	空	与下一项同时配置才切云库
`CLOUD_PUBLISHABLE_KEY`	空	云库匿名 Key
`CLOUD_OAUTH_RELAY`	空	可选的鉴权中继地址
`MSEDGE_PATH`	空	手动指定浏览器路径，优先于自动探测
`SMART_CHROME_PATH`	空	启动器注入，指向内置 `chrome-headless-shell`
`AGENT_MOCK`	空	设为 1 时用确定性脚本替代模型调用
`APP_PORT`	3100	起始端口，占用时顺延至 3109
`NEXT_DIST_DIR`	`.next`	构建输出目录，避免 EPERM 与类型污染
`SMART_INSTALL_DIR`	`%LOCALAPPDATA%\SmartSchedule`	安装目录，供自动化验收
`SMART_NO_SHORTCUT` / `NO_LAUNCH` / `NO_FIREWALL`	空	跳过快捷方式 / 启动 / 防火墙，验收用
`MOCK_AI_PORT` / `MOCK_AI_MODE`	4999 / `create`	假模型服务端口与模式（8 种）

表 A-2 环境变量（全部可选，见 `.env.example`）

A.3 源文件与行数

文件	行	职责
src/lib/types.ts	115	数据模型、优先级与状态字典
src/lib/store.ts	221	Zustand 全局状态与 29 个动作
src/lib/api-client.ts	147	前端唯一 HTTP 出口
src/lib/ai-config.ts	108	模型配置读写与 6 家预设
src/lib/app-settings.ts	62	教学周历设置与周次换算
src/lib/course.ts	62	课程识别与状态推算（双端共用）
src/lib/server/index.ts	9	本地 / 云库分发
src/lib/server/repository.ts	242	本地 JSON 仓储与自动维护
src/lib/server/cloud-repository.ts	228	PostgREST 云库适配
src/lib/server/ai-core.ts	220	模型调用、抽取提示词、错误中文化
src/lib/server/agent/runtime.ts	515	有界循环控制器
src/lib/server/agent/tools.ts	622	九个工具的 Schema 与执行器
src/lib/server/agent/log.ts	46	撤销日志落盘
src/app/api/todos/route.ts	116	列表与五分支批量端点
src/app/api/todos/[id]/route.ts	32	单条读写删
src/app/api/agent/route.ts	53	Agent 入口
src/app/api/agent/patrol/route.ts	126	今日简报与越权检测
src/app/api/agent/undo/route.ts	52	逆序回滚
src/app/api/ai/chat/route.ts	207	五种文本任务与本地兜底
src/app/api/ai/test/route.ts	55	连通性测试
src/app/api/import/parse/route.ts	424	多格式解析与双模式
src/app/api/export/route.ts	130	三种导出
src/app/api/export/image/route.ts	226	无头浏览器出图
src/app/api/auto/route.ts	17	自动维护触发
src/app/api/health/route.ts	14	应用指纹
src/app/api/network/route.ts	27	局域网地址枚举
src/app/api/qr/route.ts	30	本地生成二维码 SVG
src/components/AppShell.tsx	561	外壳、导航、搜索、批量
src/components/views.tsx	486	五个视图
src/components/AgentPanel.tsx	486	Agent 抽屉与确认卡片
src/components/AIPanel.tsx	482	五个 AI 功能页
src/components/SettingsPanel.tsx	483	设置四页签
src/components/TodoFormModal.tsx	422	新建 / 编辑表单

文件	行	职责
src/components/ImportModal.tsx	389	导入向导与双模式
src/components/RichText.tsx	231	富文本排版引擎
src/components/TodoCard.tsx	147	日程卡片
src/components/ui.tsx	128	UI 原子件
src/components/ui/Select.tsx	153	自绘下拉框
src/app/globals.css	133	主题变量与原子样式
src/app/layout.tsx	41	根布局与元信息
src/app/page.tsx	5	首页
src/lib/reminder-types.ts	56	提醒事件与设置类型
src/lib/reminder-client.ts	115	通知权限、SW 注册与弹窗
src/lib/server/data-dir.ts	10	数据目录解析（支持 SMART_DATA_DIR）
src/lib/server/reminders/engine.ts	172	提醒规则引擎（纯函数）
src/lib/server/reminders/store.ts	120	提醒队列与设置持久化
src/lib/server/reminders/scheduler.ts	88	20 秒轮询与过期清理
src/app/api/reminders/route.ts	107	提醒读取、确认、测试与设置
src/components/ReminderCenter.tsx	121	轮询与浮层
src/components/ReminderSettings.tsx	178	设置页提醒面板
src/lib/theme.ts	91	主题枚举、持久化与切换
src/components/ThemeSwitcher.tsx	101	侧栏/顶栏主题菜单
src/components/ThemeSettings.tsx	66	设置页外观面板
src/components/SeniorGuide.tsx	58	老年版三步指引

表 A-3 src/ 下 54 个文件（合计 9,454 行）

A.4 配套脚本

文件	行	用途
scripts/eval-agent.mjs	220	14 条 Agent 评测用例
scripts/mock-openai-server.mjs	187	假模型服务，8 种模式
scripts/gen-icons.mjs	13	PWA 图标生成
build-desktop/launcher.js	147	Windows 启动器（SEA 入口）
build-desktop/build-platforms.py	182	macOS / Linux 产物生成
installer/sea-installer.js	199	自解压安装器
db/schema.sql	29	云库建表脚本

表 A-4 配套脚本（不计入 src/ 统计）

文档结束

本文与源码同步维护。若代码发生结构性调整（新增存储后端、改动 Agent 协议、更换打包方式）， 对应章节需要一并更新，尤其是第 7、10、12 章中的常量与方法签名——这三处的过期信息会直接误导读者。